

Sessões de fotos, galeria e múltiplos jogos

Edição pública anonimizada - 05/10/2026. O pedido 5000 e os nomes foto-NN.jpg são exemplos fictícios; não identificam o cliente da bancada. As medições agregadas permanecem reais. Fotografias, nomes, telefones, caminhos locais e hashes individuais de mídia foram omitidos. Artefatos privados mencionados na auditoria não acompanham esta edição; consultar evidence/benchmark-summary.json e evidence/source-index.json para os recibos públicos. A documentação registra fatos e propostas, não uma integração já implantada.

Auditoria forense e arquitetura operacional recomendada

30

fotos principais

90

derivados validados

97,90%

redução do conjunto

Evdência | Caso de referência: pedido 5000 | 05/10/2026 | Versão 2.0 - três repositórios e plano de implementação

Classificação: documentação técnica pública anonimizada. Status: auditoria local concluída, otimização medida em bancada e proposta de integração; publicação de sessão, consulta real ao CRM e envio ao cliente não foram executados.

Decisão recomendada

Manter o **EvydFlow** como **orquestrador**, reaproveitar o **worker Node/Sharp** do projeto de jogos como **único responsável pelas versões web** e criar **uma sessão fotográfica persistente, com galeria e catálogo compartilhados pelos jogos**. O número final da pasta é a referência interna do pedido no CRM, preservada integralmente. A URL entregue à família usa um token aleatório independente desse número.

A galeria completa deve ser uma rota da própria gameplay, em /s/:token/fotos, reaproveitando a experiência de festive-gallery-show e os componentes PhotoPrint/SessionPhotoAlbum. O Hub em /s/:token permanece compacto; os jogos continuam em /s/:token/game/:gameId. Trata-se de um aplicativo e uma autoridade de sessão, não de dois sistemas que processam e armazenam as mesmas fotos.

Para o caso 5000, usar somente os **30 arquivos principais diretamente na raiz**. As 30 imagens de BAIXA não entram como origem do novo processamento. Galeria e jogos devem receber derivados dos mesmos originais, com os mesmos IDs de fotografia e a mesma revisão de sessão.

Resultado medido: 314,66 MB de entrada produziram 90 WebP, somando 6,62 MB, em 21,75 segundos nesta máquina. A redução do conjunto de derivados em relação aos originais foi de 97,90%. Os originais continuaram intactos. Isso demonstra a viabilidade do processamento local; não representa um teste de carga de produção ou de toda a experiência de cada jogo.

Sumário

Auditoria do caso, evidências locais e desenho do novo workflow.

1. Escopo, método e força das evidências	4
2. Inventário real da sessão 5000	5
3. Como o workflow atual realmente funciona	6
4. Achados prioritários e tratamento	8
5. O que o projeto de jogos já oferece e o que falta	9
6. Comparação das abordagens	11
7. Otimização medida e receita inicial	12
8. Identidade: pedido, sessão, fotografia, revisão e acesso	14
9. Arquitetura alvo e organização de pastas	15
10. Manifestos e chave de idempotência	16
11. Fluxo operacional ideal	17
12. Ledger, concorrência e retomada	19
13. Publicação, backend e acesso às fotos	20
14. Galeria e catálogo dos onze jogos	21
15. Dimensionamento com base no caso medido	23
16. Publicação parcial, curadoria e gates	23
17. CRM, comunicação e backup organizado	24
18. Plano de implementação por entregas	25
19. O que foi validado nesta auditoria	26
20. Mapa de evidências locais	27
21. Referências técnicas primárias	30
22. Recomendação final para o pedido 5000	30

Anexo A. Inventário individual mantido privado	31
Anexo B. Receita e fronteiras da bancada	31
PARTE II - Plano de implementação	32
P01. Resultado de produto e decisões de arquitetura	32
P02. Fronteiras dos três repositórios	33
P03. M0 - identidade e contrato comum	34
P04. M1 - worker Node/Sharp e manifesto de conteúdo	35
P05. M2 - persistência, revisões e autoridade do backend	36
P06. M3 - contrato HTTPS `prepared-derivatives`	37
P07. M4 - acesso da família e sessão real no React	38
P08. M5 - galeria mobile dentro da gameplay	39
P09. M6 - provider e flow do EvydFlow	41
P10. M7 - evento de ativação, inbox e entrega	42
P11. M8 - memória dos jogos e proteção da fotografia	42
P12. Matriz de verificação integrada	43
P13. M9 - piloto, migração e rollback	43
P14. Backlog executável, tamanho e dependências	44
P15. Condição de conclusão e entregáveis	46

1. Escopo, método e força das evidências

Foram inspecionados: a pasta fornecida do cliente; os workflows, tasks, fila, estado e integrações em <repo-evydfLOW>; os contratos de fotos/sessões, aplicativo, servidor de catálogo e pipeline de mídia em <repo-gameplay>; e o repositório privado hypeneural/festive-gallery-show, clonado para leitura e verificação na pasta reference-repos desta auditoria. O texto colado pelo solicitante foi usado como hipótese/checklist e confrontado com as fontes.

A pasta de referência é:

```
<origem-privada>\<nome> - <telefone> - Experiência Então é Natal - 5000
```

O solicitante informou que o sufixo 5000 corresponde ao ID do pedido do CRM. Isso foi adotado como regra de negócio de entrada. **A existência, o UUID, o status comercial e o destinatário desse pedido não foram consultados no CRM real.** O contrato remoto necessário para resolver um número de pedido em UUID continua sendo uma pendência de integração.

A inspeção foi estática nos sistemas operacionais. Nenhum flow de produção foi executado, nenhum original foi movido, nenhuma permissão do Drive foi alterada, nenhum upload foi feito e nenhuma mensagem foi enviada. Os arquivos reais de secrets, .env e credenciais não foram lidos. Apenas a bancada isolada de mídia e a galeria offline foram executadas.

Registro de versões

Sistema	Versão observada	Observação
Jogos	e38573b46d841f9cea1ab5f7306b0fd254b511ca	Repositório sincronizado; código de mídia reutilizado na bancada.
EvydFlow	42f0bcfd562d3165362cba39f1adb7bc5856d3af	Refresh dos achados contra este commit; outros trabalhos continuam alterando SoClique.
Galeria React	0f903fba53d0e33c62cd3aa32f1cb25e8489390b	hypeneural/festive-gallery-show, branch main; clone privado de referência.
Sharp / libvips	0.35.3 / 8.18.3	Versões efetivamente usadas no experimento.
Bancada	Duas fotos em processamento; sharp.concurrency(1)	Controle conservador de recursos, não uma configuração universal.

Na etapa inicial, o EvydFlow estava em 0c99818 com alterações locais. Outro trabalho commitou essas alterações em 42f0bcf; os achados centrais foram reconferidos, e os arquivos do scan, ID, imagens, FTP, Drive, CRM e runner legado não mudaram entre essas referências. Depois do refresh, novos edits em soclick/models.py e soclick/storage.py foram observados. Os detalhes de SoClique são referências ao snapshot/commit inspecionado, não promessa sobre futuras alterações. A auditoria não escreveu no código do EvydFlow. Snapshots, hashes e estados de captura foram preservados em evidence/source-snapshots e evidence/source-receipts.json; o recibo inicial também permanece disponível.

Neste documento, **observado** significa comportamento comprovado por código ou medição; **risco** é a consequência possível desse comportamento; **proposto** é uma alteração ainda não implementada. A classificação não afirma que houve incidente de vazamento ou falha operacional em produção.

2. Inventário real da sessão 5000

Item	Resultado observado
Origem selecionada	Arquivos diretamente na raiz da pasta do cliente.
Fotos principais	30 JPEG; 314.662.440 bytes, ou 314,66 MB.
Subpasta excluída	BAIXA: 30 arquivos, 20.638.181 bytes, ou 20,64 MB.
Orientação	25 horizontais e 5 verticais; nenhuma quadrada.
Resolução	De 7,89 a 22,40 megapixels.
Tamanho por original	De 4.069.068 a 17.486.517 bytes.
Metadados nos originais	30 com EXIF, 30 com ICC e 30 com XMP.
Orientação EXIF	Valor 1 em todos os arquivos desta amostra.
Duplicação exata	Nenhum par com o mesmo SHA-256 do arquivo original.
Candidatos a duas versões da mesma captura	foto-13.jpg e foto-12.jpg; mesmos 5425×3875 pixels, hashes distintos.

Os dois arquivos de captura semelhante devem permanecer na seleção inicial. Nomes parecidos não comprovam duplicação visual. O fotógrafo pode escolher uma versão final durante a curadoria; o sistema não deve apagar ou fundir esses arquivos automaticamente.

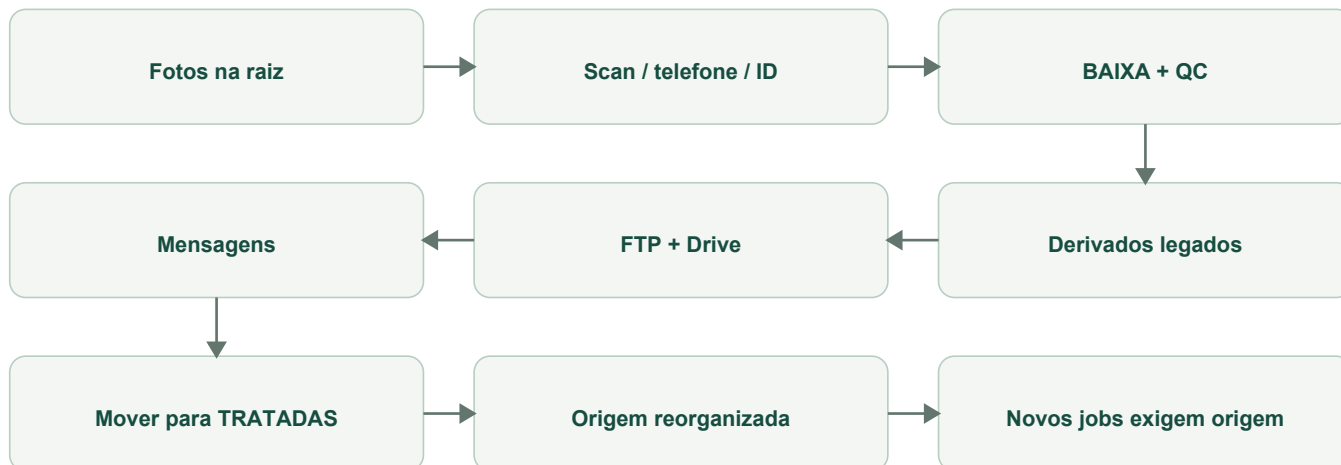
O baseline histórico em docs/media/NATAL_2024_CORPUS_BASELINE.md registra 10.244 cabeçalhos de fotos, de 396 sessões não vazias: mediana de 23 fotos, P90 de 44, máximo de 92; 63,2% horizontais e 36,8% verticais. Esse documento foi lido, não houve nova varredura do acervo nesta auditoria. Ele é evidência de variedade de tamanho/orientação, não prova de decode completo daquele acervo. A nova galeria precisa trabalhar com 1, 5, 30, 92 e fixtures de estresse maiores, sem assumir um lote fixo de 30. [E17]

Regra de seleção: não fazer busca recursiva. BAIXA, QC, TRATADAS, BRUTAS e outras subpastas não entram nesta execução. Se uma pasta futura estiver vazia porque o legado moveu os originais para TRATADAS, a origem deve ser selecionada explicitamente pelo operador e registrada. Não procurar uma alternativa silenciosamente.

Unidades: MB e GB neste relatório são decimais, com 1 MB = 1.000.000 bytes. Os JSONs preservam os valores inteiros para reprodução dos cálculos.

3. Como o workflow atual realmente funciona

O EvydFlow já tem uma estrutura útil: DAG YAML, tasks reutilizáveis, retomada por step, fila e integrações de entrega. Porém, o flow legado associa processamento, publicação, comunicação e movimentação de originais na mesma cadeia.



Resumo conceitual do legado. A saudação pode ocorrer antes da coleção pronta.

3.1 Entrada e identidade

`fs.scan_inputs` usa `base.iterdir()`: varre apenas a raiz, atendendo à exclusão de BAIXA. Contudo, o filtro de extensões depende da capitalização declarada em settings. Também há um defeito quando `ignore_prefix` é omitido: o prefixo vazio pode excluir todos os arquivos. Os flows existentes informam `calendario`, evitando esse defeito nesse caminho específico. [E01]

`fs.extract_order_id` identifica os dígitos finais, mas usa `digits[-4:]` como `order_id` e diretório remoto. `order_id_full` preserva o valor completo. Para 5000, ambos coincidem; para 105000, o valor operacional seria 5000. O teste unitário atual deliberadamente cobre o truncamento. A regra precisa mudar no novo fluxo, sem migrar silenciosamente endereços antigos. [E01, E02]

O telefone é inferido do nome ou caminho da pasta. Isso não confirma que o número pertence ao titular do pedido nem que é o destinatário autorizado no CRM. Os YAMLS atuais não fazem reconciliação do pedido numérico com o CRM. A task `orders.detail` espera um UUID; `orders.search` oferece filtros, mas o filtro remoto correto para pesquisar o número 5000 não foi comprovado. [E03]

3.2 Imagens

O código de `images.generate_outputs` aplica orientação EXIF, converte para RGB e usa LANCZOS. Os defaults de código são BAIXA com lado maior de 2500 px e JPEG qualidade 93, com logo; e QC em canvas branco de 660×660 px. **QC é um encaixe proporcional com bordas, não um recorte quadrado da fotografia.** O default permite ampliar imagens pequenas em QC. Configurações externas reais podem alterar esses valores e não foram lidas. [E04]

BAIXA e QC são criadas dentro da origem e recebem nomes baseados no stem. Isso permite sobrescritas, colisões entre extensões e resíduos de execuções anteriores. Falhas individuais são logadas e toleradas; a função pode retornar sucesso parcial sem um gate de coleção completa. Preservar o ICC original em um arquivo convertido para RGB não equivale, por si só, a converter explicitamente o perfil para sRGB. [E04]

3.3 Entrega e estado

O FTP implementado é `ftplib.FTP`, com upload direto ao nome final. Não foi encontrada implementação de `FTP_TLS` nesse caminho. A documentação que promete TLS é mais forte que o código. Os links dos flows legados usam HTTP e número do pedido. [E05]

O ramo Drive copia originais para TRATADAS e compartilha essa pasta com `type=anyone, role=writer, allowFileDiscovery=false` limita descoberta, não restringe o acesso a uma pessoa. O código cria permissão de edição para quem possui o link. Não foram verificadas permissões reais já existentes no Drive. [E06]

O estado do runner não é uma transação de publicação. O fingerprint inclui bytecode `co_code` e YAML antes da renderização, mas não o conteúdo das imagens, a receita, as configurações, o logo ou os outputs das dependências. Os arquivos de estado são escritos diretamente. O cache pode considerar um step concluído sem conferir a integridade dos artefatos atuais. [E07]

O YAML descreve dependências, mas o runner executa steps em sequência. A fila genérica não demonstra lock exclusivo por sessão/revisão. Retries de steps de envio podem repetir mensagens ou mídias após um resultado remoto desconhecido. [E08]

No flow legado, a saudação inicial depende apenas do telefone e pode acontecer antes da coleção estar pronta. O novo fluxo deve colocar a intenção de entrega após a ativação da revisão, em vez de usar uma saudação como início de uma cadeia de efeitos externos.

4. Achados prioritários e tratamento

ID	Prioridade	Achado e consequência	Tratamento recomendado
F01	P0	ID final truncado para quatro dígitos: pedidos diferentes podem compartilhar identidade e diretório.	Preservar o número completo, resolver UUID do CRM e separar sessão/token.
F02	P0	TRATADAS com originais é compartilhada como anyone/writer.	Ramo de backup privado; entrega separada com acesso de leitura definido por política.
F03	P0	O aplicativo dos jogos usa fixtures fora do modo de teste local; não hidrata a sessão real de um cliente.	Repositório persistente, API autorizada e erro explícito para sessão inválida.
F04	P0	Não há endpoint de mídia de produção implementado no servidor de catálogo auditado.	Autorizar sessão/foto/revisão/variante antes de servir o arquivo.
F05	P0	FTP sem TLS nesse código, nomes finais durante upload e links enumeráveis do legado.	Transferência autenticada e criptografada; staging, recibo e ativação atômica.
F06	P1	Fingerprint do runner não representa conteúdo e receita; estado não transacional.	Manifesto congelado, hashes semânticos e ledger transacional.
F07	P1	Sucesso parcial de imagens, raiz vazia ou derivados antigos não bloqueiam toda a entrega.	Gate de revisão completa; falhas explícitas e nenhum envio antes de READY.
F08	P1	Nomes de saída podem colidir; ordenação define IDs locais posicionais.	IDs persistentes por fotografia e namespace imutável de versão.
F09	P1	Cache Sharp atual é baseado no original, sem identidade de receita.	Incluir recipeVersion e parâmetros na chave dos derivados.
F10	P1	Sem lock genérico por sessão; retries de efeitos externos podem duplicar entrega.	Lease com fencing e outbox com reconciliação de resultado desconhecido.
F11	P1	Fluxo legado move os originais após mensagens; reexecução pode encontrar raiz vazia.	Novo flow mantém origem intacta; arquivamento é operação separada.
F12	P2	Documentação promete estado atômico, TLS e invalidação mais abrangente que o código.	Atualizar documentação junto à implementação e aos testes que comprovam as garantias.

P0 significa requisito a resolver antes da nova publicação para clientes. P1 bloqueia operação confiável e reprocessamento seguro. P2 exige correção e verificação, mas não prova incidente. Não foram alterados os flows nem as permissões durante esta auditoria.

5. O que o projeto de jogos já oferece e o que falta

Base reutilizável

O contrato `Photo` já separa ID, dimensões, proporção, orientação e URLs de thumb, card e game. Há metadados opcionais normalizados de ponto focal e zona segura do rosto, sem reconhecimento no navegador. `Session` agrupa as fotos. Os jogos importam módulos dinamicamente, com um ciclo de vida de canvas por vez. [E09]

O worker `processMediaJob` valida arquivo, limita a entrada a 32 MiB e 40 milhões de pixels, usa orientação EXIF e produz WebP proporcionais de 480, 800 e 1600 px, qualidade 82, sem ampliação. A geração ocorre em staging e usa rename para publicar um conjunto de variantes local. O batch limita a concorrência e isola falhas. [E10]

Lacunas de produção

`AppRouter` escolhe uma sessão local de teste ou `createFixtureSession`. Um endereço com token arbitrário não carrega automaticamente fotos reais. A implementação `InMemorySessionRepository` não é um banco de sessões de clientes. O servidor de catálogo entrega HTML/preview social; não implementa a API de sessão, autenticação da galeria ou endpoint das fotos. [E11]

`prepareLocalMedia` é uma ferramenta de teste: usa UUID fixo por default, token `local-private-test`, IDs definidos pela posição ordenada e uma única configuração local. É útil para bancada, mas não deve ser usado como registro multicliente. `media:inspect` foi desenhado para uma raiz de coleções de sessões; passá-lo diretamente à pasta individual pode tratar a subpasta BAIXA como sessão. O inventário desta auditoria foi explicitamente limitado aos arquivos imediatos da raiz. [E12]

O cache atual dos derivados inclui o hash do original, mas não o hash da receita. Se qualidade ou dimensões forem alteradas, um conjunto antigo ainda considerado válido pode ser reutilizado. A escrita do manifesto é atômica por arquivo, mas o merge pressupõe um escritor e pode perder atualizações concorrentes. Na cópia inicial do original, é necessário conferir o hash do snapshot copiado antes da promoção para evitar mudança da origem entre leitura do hash e cópia. São requisitos da extensão de produção, não falhas reproduzidas nas 30 fotos deste teste. [E10]

5.1 Galeria mobile real: o que portar e o que substituir

`festive-gallery-show` é um frontend React 18/Vite 5. A entrada usa `?id=<número>`, e `PhotoGallery` busca `photos.php?id=...` e `order.php?id=...`. O backend PHP desses endpoints não está no repositório clonado e não foi auditado. A ausência de autorização no frontend, por si só, não prova ausência de controles no servidor. [G01, G02]

Recurso observado	Uso no novo aplicativo
Masonry por <code>react-photo-album</code>	Portar o padrão de layout, com dimensões reservadas e adapter do contrato <code>Photo</code> .
Lightbox e plugin Zoom	Adaptar a game 1600, com navegação, foco, Escape e gestos testados.
Lote configurado: 12 mobile / 18 desktop; incremento 8 / 12	Aproveitar a estratégia, acrescentando botão manual e limite de antecipação verificável.
<code>IntersectionObserver</code> e imagens lazy/async	Manter, com fallback para indisponibilidade e sem carregamento ilimitado por sentinel.
Compartilhar/copiar link	Usar link canônico da sessão por token; não copiar o ID do CRM.
<code>Photo { src, width, height, alt }</code>	Substituir o src único pelo catálogo de variantes da sessão comum.
URLs numéricas e endpoints PHP legados	Não transportar para a rota nova; a nova fonte é a API autorizada da sessão.

Recurso observado	Uso no novo aplicativo
CSS/Tailwind global e React 18	Adaptar aos tokens, componentes e React 19 da gameplay; não copiar todo o package.json/lockfile.

O grid e o lightbox atuais usam o mesmo `photo.src`, sem variantes responsivas no contrato. Isso pode fazer uma miniatura baixar a mesma imagem grande do lightbox. Não foi comprovado qual transformação o PHP remoto efetivamente entrega; a associação com BAIXA é o caminho do flow legado, não uma inspeção do servidor. [G02]

O efeito de fetch não cancela/rejeita explicitamente respostas antigas quando `galleryId` muda. O novo provider deve ter escopo por token/sessão e descartar respostas atrasadas para impedir mistura visual entre pedidos. A implementação usa `IntersectionObserver` diretamente, sem fallback manual equivalente ao picker atual da gameplay. [G02]

Verificação do clone: `npm ci --ignore-scripts` usando package-lock e build Vite aprovados. A interface abriu fotos e lightbox, encerrou com Escape e não teve erros de página/overflow em 390, 768 e 1280 px. Todas as respostas de `photos.php`, `order.php` e mídia foram interceptadas e simuladas com derivados locais; não houve consulta de cliente nos servidores reais. O teste mediu 28 cards renderizados no mobile e 30 nos outros viewports no ponto de observação, apesar de existir configuração inicial de lote; o orçamento real de render/carregamento deve ser testado, não deduzido apenas da variável `initialBatchSize`.

As dependências instaladas `react-photo-album 3.2.1` e `yet-another-react-lightbox 3.25.0` declaram React 19 em `peer dependencies`. Isso torna o port plausível, mas não comprova integração com o monorepo. O build antigo também apresentou aviso de base `Browserslist` antiga. Não foram atualizadas dependências nem executados testes de um port ainda inexistente. [G03]

5.2 Base existente de galeria na gameplay

`SessionPhotoAlbum` já oferece foto escolhida, anterior/próxima, swipe e picker em dialog com 12 fotos e botão para mais 12. `PhotoPrint` usa `card` para a foto do álbum e `thumb` para o picker; reserva proporção e estados de `loading/error`. A intenção declarada é manter a coleção completa fora do início do Hub para não empurrar os jogos para baixo. [E18]

Manter esse Hub compacto e criar uma tela `SessionGallery` dedicada. A família pode escolher uma foto na galeria/lightbox e usar a ação "Brincar com esta foto" sem perder a seleção no retorno aos jogos. Há uma regra genérica `.photo-card img { object-fit: cover }` e uma específica do picker com `contain`; adotar `fit` explícito e testado para evitar regressão de enquadramento. [E19]

6. Comparação das abordagens

Abordagem	Vantagem	Limitação	Decisão
Ampliar apenas BAIXA/QC em Pillow	Mantém um runtime Python e o legado.	Duplica a receita web existente; preserva acoplamentos e requer nova estrutura de variantes/manifestos.	Manter para entrega legada, sem fazê-la origem dos novos jogos.
EvydFlow orquestra worker Node/Sharp existente	Reutiliza contratos e processamento já medidos; mesma foto serve vários jogos.	Exige adapter, identidade de receita, ledger e backend de sessão.	Recomendado para a primeira versão.
Enviar originais e processar na VPS	Centraliza processamento.	Transferência e armazenamento maiores; transforma fotos no ambiente de hospedagem.	Não necessário para começar; avaliar somente se o volume/operacional justificar.
Adotar serviço externo de transformação	Pode delegar operação de mídia.	Introduz dependência, envio de dados de cliente e avaliação de custos/contratos.	Não introduzir nesta fase sem uma necessidade comprovada.

A recomendação é uma inferência de engenharia baseada no código disponível e na medição desta pasta. Não houve comparação de preços de fornecedores nem avaliação contratual externa.

Processar no estúdio ou na VPS?

O texto colado posiciona Sharp na VPS após o upload. Isso é uma alternativa válida, mas não é a escolha inicial recomendada após esta bancada. **Para a primeira versão, gerar os derivados no estúdio com o mesmo worker versionado e transferir somente os 6,62 MB por HTTPS**, em vez de enviar 314,66 MB de originais para a plataforma. O backend valida o conteúdo recebido, staging e recibos. Isso reduz transferência e mantém as fontes no domínio privado do estúdio.

Um modo futuro source-ingest, com Sharp no backend, pode usar o mesmo pacote e os mesmos contratos, se operação distribuída, disponibilidade do computador ou atualização do worker justificarem. Não habilitar ambos como caminhos independentes para o mesmo build. O MVP usa prepared-derivatives; o modo de origem deve ser uma decisão versionada com quotas, limites e custo medidos. O backend não pode recalcular o hash do original se só recebeu derivados: o hash de origem é proveniência do produtor, enquanto o recibo remoto comprova os bytes efetivamente recebidos.

Separação de responsabilidades: EvydFlow administra pedido, fila, retomada e entrega. O pacote de mídia administra decode, orientação, cor, resize, compressão e validação técnica. O backend de experiências administra sessão, revisões, autorização, galeria e catálogo. Cada jogo recebe um GameContext já autorizado; não consulta CRM nem abre pastas do Windows.

7. Otimização medida e receita inicial

PESO DOS ARQUIVOS - MESMAS 30 FOTOS



Escala linear. MB decimais. Derivados thumb/card/game; originais privados preservados.

7.1 Resultado com a receita existente

Versão	Lado maior máximo	Quantidade	Bytes totais	Média por foto	Maior arquivo
thumb	480 px	30	768.956	25,63 KB	39,73 KB
card	800 px	30	1.590.648	53,02 KB	87,38 KB
game	1600 px	30	4.256.622	141,89 KB	232,59 KB
Todas	Três variantes por foto	90	6.616.226	220,54 KB por foto, somando variantes	-

Processamento fresco: **21,75 s**, 30 prontas, zero falhas. Reexecução com o mesmo conteúdo/receita: **1,72 s**, zero derivados regenerados ou alterados. Todos os 90 arquivos foram decodificados na verificação posterior. Não havia EXIF nem XMP nas saídas, e todas foram identificadas como sRGB. A proporção foi conferida com tolerância de 0,005 para arredondamento de pixels. A confirmação de integridade dos originais usou SHA-256, não somente tamanho/mtime.

Esses tempos pertencem a esta máquina, com duas fotos concorrentes e libvips configurado para uma thread. Não são promessa de tempo na VPS, em outro computador ou em um lote de centenas de clientes.

7.2 Compressão alternativa: amostra de três fotos

Foram usadas a maior foto em bytes, a foto na posição intermediária da ordenação e a menor foto em bytes: foto-09.jpg, foto-16.jpg, foto-26.jpg. Todas foram comparadas com referência da origem redimensionada para o mesmo limite de 1600 px em sRGB.

Formato / qualidade	Média do arquivo	Tempo médio de transformação	PSNR médio
WebP 78	125,74 KB	0,98 s	37,02 dB
WebP 82	147,94 KB	1,07 s	37,79 dB
WebP 86	182,64 KB	0,93 s	38,59 dB
AVIF 45, effort 4	65,54 KB	10,59 s	36,28 dB

Decisão inicial: WebP 82. Já está integrado e oferece um equilíbrio demonstrado neste corpus. AVIF 45 produziu cerca de 55,70% menos bytes que WebP 82 nas três amostras, mas levou aproximadamente 9,94 vezes mais tempo de transformação e apresentou PSNR inferior. Não comparar diretamente números de qualidade entre codecs. A variação entre tempos de WebP indica que a execução única, sem isolamento de cache/carga, não é um ranking rigoroso de velocidade.

PSNR é um indicador matemático, não aprovação visual de pele, olhos, cabelo, roupa ou iluminação de Natal. A escolha final deve ser revisada em fotos completas e detalhes a 100%, em aparelhos reais. WebP 86 pode ser uma exceção para foto hero que precise de mais detalhe; isso deve formar uma receita/versionamento próprio, não sobrescrever WebP 82.

7.3 Regras de processamento

1. Ler o snapshot imutável da foto principal, nunca o JPEG já reduzido de BAIXA.
2. Validar extensão normalizada, formato decodificado, dimensões, canais e limites.
3. Normalizar orientação; converter cores para sRGB; não propagar EXIF/XMP dos originais.
4. Redimensionar proporcionalmente com `fit: inside` e `withoutEnlargement: true`.
5. Gerar as três variantes diretamente da mesma origem, evitando recompressão em cadeia.
6. Guardar dimensões finais, MIME, bytes e SHA-256 de cada saída.
7. Preservar fontes no armazenamento privado do estúdio; gerar branding de interface separadamente. Marca d'água nos pixels, se exigida, deve ter receita própria e aprovação visual.

Os comportamentos de `resize` e `remoção de metadados` foram confrontados com as referências oficiais do Sharp. [S01, S02]

Uma variante de galeria ampliada de 2048 px pode ser adicionada posteriormente se o zoom justificar. Ela não foi medida no lote completo nem faz parte dos 6,62 MB reportados. Não é necessário criar uma cópia da foto para cada jogo.

8. Identidade: pedido, sessão, fotografia, revisão e acesso

Identificador	Função	Regra
crmOrderNumber	Número final da pasta, como 5000.	Preservar todos os dígitos; não truncar nem usar como senha.
crmOrderUuid	Identidade canônica retornada pelo CRM.	Resolver e confirmar correspondência única.
photoSessionId	Sessão fotográfica persistente.	UUID gerado uma vez e reutilizado nas reexecuções.
photoId	Identidade estável da foto selecionada.	Persistente, sem nome/telefone; não mudar ao inserir outra foto antes dela na ordenação.
sourceHash	Conteúdo do snapshot original.	SHA-256 completo dos bytes; alteração de conteúdo cria nova versão.
recipeKey	Como os derivados foram produzidos.	Hash de parâmetros explícitos, versão e contrato de mídia.
revisionId	Coleção publicada e imutável.	Identifica seleção, ordem, capa, fotos e variantes de uma revisão.
accessToken	Credencial do link da família.	Aleatório e independente do pedido; expiração/revogação próprias.
jobId / runId	Execução operacional.	UUID; não substituir a identidade da sessão.

Regra de parsing proposta

O parser deve considerar o nome da pasta, com sufixo ancorado no separador esperado: - <dígitos>. Para esta pasta, o resultado é a string 5000. Preservar zeros à esquerda até o contrato do CRM definir a canonicalização. Não aplicar conversão para inteiro como única representação.

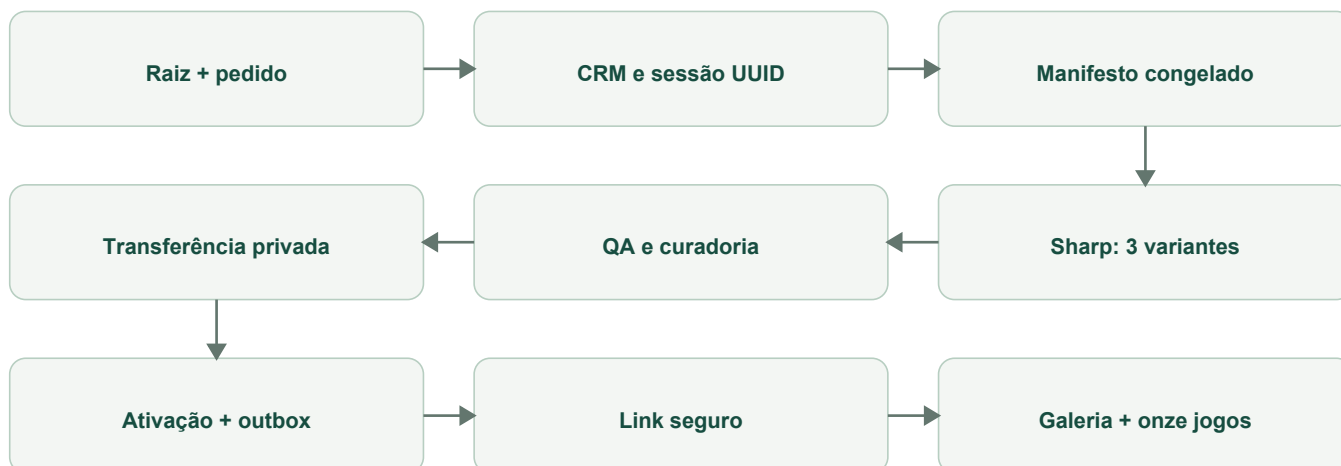
Exemplos de resultado: 5000 -> 5000; 105000 -> 105000; 1004821 -> 1004821. Nome sem sufixo, pedido ambíguo, não encontrado ou incompatível com o cadastro deve bloquear a publicação e abrir tarefa de conferência. O telefone extraído da pasta serve como pista para reconciliação, nunca como autorização autônoma de envio.

Um pedido pode ter mais de uma sessão ou campanha. Por isso, a unicidade deve incluir o estúdio/tenant, a identidade canônica do CRM e a sessão fotográfica. Se uma nova pasta indicar o mesmo pedido com conjunto diferente de fotos, o operador deve escolher entre revisão da sessão existente e nova sessão. Não juntar lotes nem substituir a galeria silenciosamente.

Foto repetida em vários jogos

A mesma fotografia conserva o mesmo photoId. Cartas duplicadas na Memória, ocorrências repetidas na chuva de Rudolph e peças de um quebra-cabeça são objetos do jogo, não novos arquivos de mídia nem novas fotos do catálogo. O álbum de resgate usa identidade única da foto; repetições podem carregar magia sem duplicar a galeria.

9. Arquitetura alvo e organização de pastas



Fluxo proposto. Nenhuma task de publicação foi executada nesta auditoria.

No computador do estúdio - estrutura proposta

```

<workspace>\
  evydf\          código do orquestrador
  secrets\        credenciais externas, fora do Git
  data\photo-experiences\
    ledger\experiences.sqlite  banco local operacional, disco local
  orders\<crm-namespace>\5000\<session-uuid>\
    intake\          vínculo com a origem e inventário privado
    snapshots\<source-hash>\  snapshot/cache privado, sem mover a origem
    builds\<build-fingerprint>\
      staging\        derivados ainda não publicáveis
      media\<photo-id>\<recipe-key>\thumb.webp
      media\<photo-id>\<recipe-key>\card.webp
      media\<photo-id>\<recipe-key>\game.webp
      private-manifest.json
      delivery-manifest.json
      validation-receipt.json
    receipts\        publicação, CRM e notificações
  data\audits\      relatórios e bancadas isoladas
  
```

A pasta do cliente continua intacta. Nome, telefone e caminho completo são necessários apenas no registro privado de intake. Não aparecem na URL das fotos, no manifesto entregue ao navegador ou na estrutura pública de assets dos jogos. O layout de produção é uma proposta, não uma descrição de diretórios já implementados.

No backend dos jogos - estrutura proposta

```

application/releases/<release-id>/  app e assets comuns aos jogos
private-media/<session-uuid>/<revision-id>/<photo-id>/...
database/                          sessões, permissões, revisões e catálogo
staging/<session-uuid>/<revision-id>/ material recebido ainda não ativo
  
```

Os assets do jogo são comuns a todos os clientes. Fotos do cliente ficam fora da webroot. Não colocar cada pedido em uma build própria do aplicativo, em public/assets ou no GitHub. A atualização de um jogo não deve exigir reprocessar fotos cujas receitas permaneceram iguais.

O banco local do Windows e o banco do backend são autoridades diferentes. Eles trocam recibos por uma API autenticada; não compartilham um arquivo SQLite em Drive, SMB ou HD de rede. SQLite/WAL é uma opção para uma autoridade em um host, com transações curtas e um escritor por vez; PostgreSQL pode ser avaliado se surgirem múltiplos hosts ou contenção relevante. [S03]

10. Manifestos e chave de idempotência

Manifesto privado

Deve registrar tenant, pedido completo/UUID CRM, UUID da sessão, origem autorizada, hashes originais, estado da curadoria, receita e recibos. Pode conter caminhos locais e nomes de arquivo; fica restrito ao estúdio/backend administrativo.

Manifesto de entrega

Deve conter somente os dados necessários ao navegador: versão, sessão opaca, revisão, nome de exibição aprovado, fotos autorizadas, dimensões/proporção, URLs de variantes, capa/ordem e jogos habilitados. Não deve conter telefone, CPF, UUID do CRM, diretórios Windows, original ou credenciais administrativas.

Exemplo de projeção proposta, sem token real:

```
{
  "schemaVersion": 1,
  "sessionId": "uuid-da-sessao",
  "revisionId": "uuid-da-revisao",
  "displayName": "Seu Natal em família",
  "coverPhotoId": "ph_opaco_01",
  "enabledGameIds": ["puzzle-swap", "memory", "rena-das-lembrancas"],
  "photos": [
    {
      "id": "ph_opaco_01",
      "width": 5600,
      "height": 4000,
      "aspectRatio": 1.4,
      "orientation": "landscape",
      "variants": {
        "thumb": "/api/media/ph_opaco_01/thumb?r=uuid-da-revisao",
        "card": "/api/media/ph_opaco_01/card?r=uuid-da-revisao",
        "game": "/api/media/ph_opaco_01/game?r=uuid-da-revisao"
      }
    }
  ]
}
```

O contrato existente `session` tem `publicToken` obrigatório. A implementação deve definir uma adaptação explícita na borda de acesso/compartilhamento, em vez de substituir o campo por número do pedido ou propagar `secrets` em manifestos permanentes. `enabledGameIds`, `revisão` e `capa` são extensões propostas, ainda ausentes nesse contrato atual.

Fingerprint semântico

```
recipeKey = SHA256(JSON canônico de:
  recipeVersion + limites + formato + qualidade + orientação + política de cor
  + políticas de metadados + versão do worker/encoders + contrato de mídia)

assetKey = SHA256(sourceHash + recipeKey)

buildFingerprint = SHA256(JSON canônico de:
  tenant/session + photoIds e hashes estáveis + ordem/seleção/capa
  + recipeKeys + versão de curadoria/contrato)
```

Não incluir `jobId`, `timestamp` ou caminho do Desktop no hash semântico. Esses dados pertencem ao snapshot de proveniência. Alterar somente a capa/ordem deve criar nova revisão de catálogo sem recomprimir arquivos idênticos. Alterar qualidade ou encoder deve criar derivados em outro namespace. Inserir uma foto não deve renumerar todas as demais.

Para cada imagem: copiar a origem para snapshot temporário, recalculando SHA-256 dos bytes copiados, comparar com o manifesto congelado e só então promover. Validar bytes/hash/dimensões de outputs reutilizados. Um path ou mtime igual não comprova conteúdo igual.

11. Fluxo operacional ideal

Etapa	Entrada e responsabilidade	Saída / gate
1. Registrar intake	Operador escolhe a pasta; allowlist e boundary de paths.	Intake privado; nenhuma busca recursiva.
2. Validar origem	Raiz, formatos suportados, decode, tamanho/pixels, estabilidade e ausência de links que escapem da origem.	Inventário; zero fotos bloqueia.
3. Resolver pedido	ID completo da pasta; CRM retorna uma única identidade canônica.	Pedido/sessão e destinatário reconciliados.
4. Congelar seleção	Ordem, fotos aprovadas, capa e hashes completos.	Manifesto FROZEN com fotografia estável por ID.
5. Derivar	Adapter Python chama worker Node/Sharp.	Conjunto versionado em staging; falhas identificadas.
6. Conferir	Contagem, variantes, decode, proporção, metadata, orçamento e hashes.	VALIDATED; por default todas as fotos selecionadas prontas.
7. Revisar experiência	Capa, 25 horizontais/5 verticais, enquadramento e seleção por jogo.	Curadoria aprovada; catálogo mínimo atendido.
8. Transferir	Somente derivados e manifesto de entrega por canal seguro autenticado.	Recibo remoto com conjunto esperado e hashes.
9. Ativar	Backend valida e troca ponteiro de revisão em transação.	ACTIVE, sem expor um lote pela metade.
10. Agendar entrega	Outbox na transação de ativação.	Evento pronto, sem enviar durante compressão.
11. Comunicar	Destinatário CRM confirmado; dispatcher e recibo do provedor.	Sent/Unknown/Failed, com reconciliação.
12. Arquivar	Política de backup/retenção separada, se autorizada.	Fontes continuam rastreáveis; nenhuma exclusão implícita.

Adapter Python - Node

Usar processo filho com `shell=False`, executável e versão fixados, JSON de job via stdin/arquivo privado e schema validado. Não montar comandos concatenando o nome da pasta, telefone ou campos do CRM. stdout deve ter resultado estruturado; stderr e eventos de progresso ficam em logs redigidos. Definir timeout, cancelamento e retorno não zero em erro determinístico.

O pacote de mídia fica sob responsabilidade do projeto de jogos, mas deve possuir uma distribuição/CLI versionada para o worker do estúdio. Um serviço instalado não deve depender permanentemente de uma pasta pessoal no Desktop ou de `pnpm dev`. EvidFlow registra a versão exata do worker usado no recibo de build.

DAG proposto - não executar como flow existente

```
name: photo_experience_publish # PROPOSTA: tasks ainda precisam ser implementadas
steps:
  - id: intake
    uses: experiences.validate_intake
  - id: order
    needs: [intake]
    uses: orders.resolve_session
  - id: freeze
    needs: [intake, order]
    uses: media.freeze_manifest
  - id: derivatives
    needs: [freeze]
    uses: media.generate_game_gallery
  - id: quality
    needs: [derivatives]
    uses: media.validate_revision
  - id: publish
    needs: [quality]
    uses: experience.publish_revision
  - id: ready_notification
    needs: [publish]
    uses: notifications.enqueue_ready
```

O novo flow deve ser independente de `natal_default`, `natal_laboratorio` e `natal_massivo`. Alterações nesses flows podem enviar mensagens ou mover originais; não devem ser acionadas automaticamente como efeito da criação da galeria.

Interface do operador

Proposta para a primeira versão: uma ação explícita no painel/menu de contexto, como "Preparar galeria e jogos", recebe a pasta e mostra pedido, origem, quantidade, fotos candidatas e estado. O operador confirma vínculo com o CRM e curadoria; depois acompanha build, validação, publicação e entrega em colunas separadas. Um watcher automático pode ser adicionado depois, com debounce, confirmação de pasta concluída e unicidade de job. Criar um arquivo novo durante a exportação do fotógrafo não deve publicar uma sessão por acidente.

12. Ledger, concorrência e retomada

Estados propostos:

```
DISCOVERED -> CRM_RESOLVED -> FROZEN -> DERIVING -> VALIDATED
-> STAGED_REMOTE -> ACTIVE -> DELIVERY_PENDING -> DELIVERED
```

Estados de exceção: BLOCKED, RETRYABLE_FAILURE, FAILED, SOURCE_CHANGED, UNKNOWN_OUTCOME, REVOKED, EXPIRED.

Um estado pronto precisa de recibo e artefatos verificáveis. `done.flag` não é suficiente. Propor unicidade operacional por (`photoSessionId`, `buildFingerprint`, `operation`) e um lease com heartbeat por sessão/revisão. Cada commit deve verificar também o fencing token do dono; worker atrasado cujo lease venceu não pode ativar uma revisão.

Não manter a transação SQLite aberta durante decode, compressão ou upload. Fazer claim/commit curtos. Começar com um job de cliente por worker e duas fotos em paralelo no Node; aumentar depois de medir memória, CPU, I/O e latência das outras atividades do computador. `WORKER_CONCURRENCY` declarado em settings não demonstra que o loop atual use esse valor.

Padrões úteis do SoClique

O módulo SoClique tem manifesto congelado, separação de hash semântico e snapshot local, ledger SQLite/WAL, claim transacional, comparação de estado e reconciliação de resultado desconhecido. São bases melhores que o cache genérico de steps para esse novo domínio. [E13]

Não copiar sem revisão: a detecção de mutação da fonte pode depender de tamanho/mtime; o checker é opcional; há fallback destinado a fakes que pode confirmar sem metadados remotos completos; e as transições de item precisam aplicar fencing do dono do lease. O novo gate deve exigir inventário remoto autoritativo e conjunto exato de hashes.

Reexecução por cenário

Cenário	Comportamento esperado
Mesmo manifesto e mesma receita	Reutilizar derivados verificados; não enviar novamente o link sem uma nova intenção de entrega.
Nova qualidade/dimensão	Nova <code>recipeKey</code> ; gerar namespace novo, manter revisão ativa até a nova aprovação.
Inclusão de foto	Reutilizar IDs e outputs antigos; derivar apenas a nova foto; nova revisão.
Foto substituída	Novo <code>sourceHash</code> e saída; histórico permanece rastreável.
Remoção por curadoria	Tirar do novo catálogo; não ressurgir por resíduo de diretório.
Remoção por privacidade	Revogar também acesso a revisões antigas conforme política; não manter acesso por graça normal.
Falha no upload	Retomar staging por hash e recibo; revisão ativa anterior permanece.
Timeout após envio	Marcar <code>UNKNOWN_OUTCOME</code> e reconciliar com provedor; não reenviar cegamente.
Origem muda durante processamento	Bloquear a promoção do build; criar novo snapshot/manifesto após conferência.

13. Publicação, backend e acesso às fotos

Publicação de uma revisão

Transferir por protocolo criptografado/autenticado, como SFTP ou API HTTPS de ingestão privada. A escolha exata depende do ambiente de implantação, não auditado aqui. FTP público do legado não constitui o canal recomendado.

O backend recebe um namespace novo em staging; verifica todos os arquivos e hashes; registra a revisão validada; troca `activeRevisionId` em transação; e cria o evento da outbox na mesma transação. Ativar primeiro e escrever o manifesto depois expõe coleção inconsistente. Renomear diretório só é atômico sob as garantias do filesystem correspondente; para atravessar volumes, usar staging no destino e promoção local.

API proposta

Rota / operação	Responsabilidade
Link <code>/s/<token-aleatorio></code>	Entrada da família, sem número do pedido/nome/telefone na URL.
Rota <code>/s/<token-aleatorio>/fotos</code>	Galeria completa na gameplay; mesma sessão, autorização, revisão e seleção de foto.
Exchange de acesso	Verifica token, status, validade e revogação; cria cookie seguro de acesso à sessão.
GET <code>/api/sessions/current</code>	Projeta somente a revisão autorizada e catálogo permitido.
GET <code>/api/media/<photoId>/<variante>?r=<revisionId></code>	Autoriza sessão, foto, revisão e variante; serve somente derivados.
Ingestão administrativa	Autenticada por credencial de serviço; recebe manifesto/arquivos, nunca pública.
Ativação administrativa	Exige recibo válido, dono/lease correto e condição de concorrência.

Essas rotas de API são propostas, não endpoints existentes no código auditado. O backend precisa ter schemas, testes de autorização e limites para elas.

Token e autorização

Recomendação: gerar token com CSPRNG, por exemplo 32 bytes aleatórios em Base64URL; armazenar representação hash, não tokens em logs. O acesso por link é uma credencial de posse, e não comprovação individual da identidade de cada visitante. Expiração, revogação e eventual confirmação adicional precisam refletir a política do estúdio.

Usar cookie `Secure`, `HttpOnly` e `SameSite` apropriado à navegação. Negar por default. Toda requisição de foto deve verificar a relação entre sessão, fotografia, revisão e variante. IDs difíceis de adivinhar não substituem autorização. [S04]

WhatsApp e outros crawlers podem abrir o link para gerar preview. Não consumir o token permanentemente no primeiro GET automatizado. O preview social deve usar arte genérica do estúdio por default; a foto de uma família em preview externo precisa de decisão específica. A troca de acesso para navegação humana pode ser uma operação separada.

Servir arquivos: não confundir o proxy

O catálogo atual emite `X-Accel-Redirect` para preview, compatível com uma configuração interna de Nginx. **Esse header não cria sozinho um endpoint de fotos nem autorização.** É necessário um proxy configurado para consumir o redirecionamento interno.

Para uma primeira versão simples, o backend pode autorizar e transmitir o arquivo por streaming sob o proxy TLS existente. Alternativamente, pode usar Nginx com `internal` e X-Accel depois da autorização. Caddy oferece interceptação configurável de resposta, mas não se deve assumir automaticamente a semântica de X-Accel sem implementar e testar a configuração correspondente. Nenhuma configuração de VPS foi modificada ou validada nesta auditoria. [E14, S05]

Cache e revogação

Separar assets públicos dos jogos de fotos privadas. Assets comuns versionados podem ter cache longo. HTML, manifestos de sessão e respostas de acesso devem evitar cache compartilhado. Como baseline conservador para fotos de cliente, usar `private`, `no-store` e reaproveitar apenas recursos em memória na execução atual; se o desempenho justificar cache privado com TTL curto, documentar o prazo e o efeito na revogação.

Não persistir fotos de cliente em cache público/CDN ou service worker indiscriminado. Revogar acesso impede novas requisições, mas não retira screenshots ou cópias já obtidas por visitantes autorizados. Não registrar token, telefone, path de origem ou URL completa em analytics dos jogos.

14. Galeria e catálogo dos onze jogos

Experiência da família

Um único link abre a galeria e o hub. A galeria mostra miniaturas, abre a foto inteira e permite selecionar a foto em destaque. Ao escolher um jogo de foto única, a seleção é preservada. Ao sair, a família retorna à mesma sessão; não precisa receber onze links ou fazer upload novamente.

<code>/s/<token></code>	Hub compacto
<code>/s/<token>/fotos</code>	Galeria completa
<code>/s/<token>/game/<game-id></code>	Jogo da mesma sessão

`AppNavigation` ainda não declara `GalleryRoute`, e `CatalogServer` ainda não aceita `/fotos` no parser de HTML/social. É necessário mudar os dois, inclusive acesso direto/reload, back e confirmação de saída durante um jogo. Não basta acrescentar um botão que aponta para uma URL atualmente interpretada como Hub ou rejeitada pelo catálogo. [E20]

A coleção completa de 30 fotos continua disponível na galeria. Cada jogo usa uma seleção adequada ao seu contrato. Metadados `recommendedPhotos` são sugestões de catálogo; não devem ser tratados como prova de toda regra interna do runtime.

Jogo / pacote	Mínimo	Recomendações	Seleção	Consumo de mídia / integração
A Magia da Minha Foto / <code>magic-photo</code>	1	1	Única	Foto hero em game.
Quebra-cabeça / <code>puzzle-swap</code>	1	1	Única	Foto escolhida em game; peças são recortes em runtime.
Memórias de Natal / <code>memory</code>	4	4	Subconjunto	Fotos em card; cartas duplicadas não duplicam os arquivos.
Trinca / <code>tic-tac-toe</code>	1	2	Subconjunto	Fotos em card, seletor usa thumb.
Expresso / <code>expresso-das-fotos</code>	1	6	Subconjunto	Seleção da mesma sessão; garantir IDs/URLs autorizados no adapter.
Guirlanda / <code>guirlanda-das-lembrancas</code>	1	6	Subconjunto	Runtime carrega game para as fotos selecionadas.

Jogo / pacote	Mínimo	Recomendadas	Seleção	Consumo de mídia / integração
Mosaico / mosaico-em-queda	1	4	Subconjunto	Fotos da mesma revisão; verificar orçamento da seleção.
Rudolph / rena-das-lembrancas	3	8	Subconjunto	Seleciona até 8 únicas com âncora; carrega card e game sob demanda.
Globo / globo-das-lembrancas	1	1	Única	Foto hero em game.
Estilingue / estilingue-das-lembrancas	1	1	Única	Prefere game, com fallbacks de variantes no runtime.
Lanterna / lanterna-magica	1	1	Única	Foto hero da mesma sessão; QA visual antes de habilitar em produção.

O pacote dev-smoke não é uma experiência para clientes. Os mínimos/recomendadas foram extraídos dos arquivos `src/definition.ts`; detalhes de consumo citados foram conferidos nos loaders de runtime. Não houve jornada completa de cada jogo com esta pasta durante a auditoria.

Foto em destaque e orientação

As 25 fotos horizontais precisam permanecer horizontais, mesmo em celular vertical. Usar proporção original e enquadramento seguro no frame; barras ou decoração do jogo podem completar o espaço, sem esticar a foto. A fotografia completa da galeria/hero deve usar `contain`, salvo recorte explicitamente aprovado. Ponto focal e área segura precisam ser relativos à imagem já normalizada por orientação, com rastreo da versão da foto.

Não usar identificação biométrica automática para definir capa. O fotógrafo aprova capa, ordem e, quando necessário, zona de proteção das pessoas. Miniatura recortada não autoriza recortar a imagem completa do jogo.

Carregamento e memória

Carregar thumbnails visíveis e antecipar poucos vizinhos; o lote de 30 thumbs mede 0,77 MB, mas não deve bloquear a primeira tela. Na abertura da foto, buscar game selecionada. Importar somente o runtime do jogo escolhido. Libertar canvas, texturas e áudio ao sair.

Memória decodificada não é o tamanho do WebP: uma imagem de 1600×1600 em RGBA usa cerca de 10,24 MB de pixels, antes de cópias/mipmaps. Não carregar todas as fotos máximas na GPU para um jogo de cartas. Fixar `revisionId` durante a rodada evita que uma publicação nova troque as fotos no meio da Memória ou do álbum de Rudolph.

A habilitação dos jogos deve ser configurável por sessão/contrato comercial e por aprovação de QA. O catálogo instalado não é, sozinho, prova de maturidade de produção de cada experiência.

Oportunidade específica na Guirlanda

O runtime seleciona até seis fotos e carrega game para todas. Expresso usa game para a âncora e card para as demais; Rudolph promove fotos de card para game conforme necessário; Mosaico planeja thumb/card/game por papel. Aplicar esse padrão à Guirlanda: uma hero em game, outras em card, promoção sob demanda e liberação segura. [E21]

Se seis fotos tiverem o mesmo aspecto e card tiver metade do lado de game, a área de pixels do conjunto cairia de seis unidades para 2,25 unidades, redução teórica de 62,5%, antes de cópias/mipmaps. É uma estimativa geométrica de memória, não medição de FPS ou prova de que todo dispositivo atingirá esse ganho. Deve ser uma entrega de otimização separada, depois das dependências de sessão/publicação.

15. Dimensionamento com base no caso medido

Cenário hipotético	Fotos por cliente	Originais	Três derivados por foto
Um cliente, caso 5000	30	314,66 MB	6,62 MB
600 clientes iguais à amostra	18.000 fotos totais	188,80 GB	3,97 GB
800 clientes iguais à amostra	24.000 fotos totais	251,73 GB	5,29 GB

São projeções lineares, não inventário da temporada nem capacidade garantida do servidor. Não incluem versões antigas, backups, logs, upload temporário, variante 2048, assets/áudio dos jogos, banco, protocolo ou margem de segurança.

Com o mesmo tempo observado e processamento sequencial por cliente, 600 lotes corresponderiam a aproximadamente 3h37 e 800 a 4h50 de geração fresca. A estimativa não inclui curadoria, hashing de intake adicional, filas de outros serviços, transferência e conferência no destino. Aumentar concorrência pode piorar tempo e memória; medir antes de alterar.

A 10 Mbit/s ideais, transferir todo o conjunto de 6,62 MB de mídia levaria cerca de 5,29 s apenas de payload. A primeira tela deve consumir uma fração disso: por exemplo, seis miniaturas da média desta sessão equivalem a aproximadamente 154 KB de fotos, além do aplicativo e outros recursos.

Organização de armazenamento recomendada: manter originais e backup no domínio privado do estúdio, publicar só os derivados necessários para jogos e galeria. O worker atual copia originais para seu cache privado; esse custo local permanece, embora os 6,62 MB representem apenas as saídas web. A bancada desta auditoria contém esse cache de originais e não é uma pasta de publicação.

16. Publicação parcial, curadoria e gates

Baseline: todas as fotos aprovadas no manifesto precisam ter as três variantes validadas. Se uma das 30 falhar, o build fica bloqueado; a revisão ativa anterior permanece. O fotógrafo pode excluir conscientemente a imagem falha e aprovar uma nova seleção de 29, mas isso cria outro manifesto/revisão e um recibo de decisão. Não converter erro em omissão silenciosa.

Gates técnicos propostos:

- Contagem entre seleção e outputs: 30 fotos e 90 variantes no caso atual.
- Decode real de todos os outputs, MIME correto, bytes e SHA-256 completos.
- Sem EXIF/XMP de origem; sRGB; orientação normalizada; sem ampliação.
- Proporção preservada e IDs sem colisão; capa pertence à coleção autorizada.
- Orçamento inicial sugerido para um lote de 30 semelhante a esta amostra: até 1 MB de thumbs, 2 MB de cards e 6 MB de game. Esses limites são proposta operacional, não requisito universal; corpus mais complexo pode demandar exceção aprovada ou revisão de receita.
- Todos os jogos habilitados atendem ao mínimo de fotos e às políticas de seleção.
- Validação da interface nos perfis móveis do projeto e em Safari/Chrome reais antes da entrega comercial.
- Nenhum original, path Windows, telefone ou segredo no manifesto do navegador.
- Nenhum arquivo de outra sessão acessível com o cookie do cliente.
- Inventário remoto e recibo completos antes de ativar e enfileirar a notificação.

Os gates de qualidade dos jogos existentes devem ser mantidos. Esta bancada prova o processamento e a galeria técnica offline, não o acabamento completo, animação, áudio, acessibilidade ou FPS de todos os onze jogos.

17. CRM, comunicação e backup organizado

CRM

Resolver o número completo para o UUID correto e confirmar estúdio, titular/destinatário, pacote e status necessário à entrega. Campos de URL da galeria/jogos, status e histórico devem usar o contrato real da API. Não inventar nomes de campos ou assumir que a busca remota aceita `order_id=5000`; isso precisa de contrato, fixture de resposta e teste de adapter.

Se o CRM estiver indisponível, preparar mídia pode ser permitido em modo local, mas publicar/vincular/enviar precisa permanecer bloqueado até reconciliar a identidade. Registrar uma causa operável, por exemplo `CRM_MATCH_REQUIRED`, para o operador não precisar reconstruir o lote.

Notificação

Separar compressão de envio. Criar outbox na ativação e verificar destinatário antes do despacho. Uma mensagem pode conter o link único da sessão, com acesso à galeria e aos jogos. Galeria pronta e mensagem enviada são estados distintos.

Guardar chave de idempotência de intenção de envio, ID do provedor, tentativas e resultado. Timeout de rede após envio é `UNKNOWN_OUTCOME`; reconciliar, não presumir falha e reenviar. Sem suporte idempotente do provedor, não prometer entrega exatamente uma vez.

Drive e originais

Backup de originais deve ter permissões privadas. Entrega de arquivos para download, quando contratada, é outro produto/ramo com destinatário e permissão explícitos. `anyone/writer` não deve ser o default do novo fluxo. Um link apenas de leitura ainda é um link compartilhável; não equivale a autenticação individual. Os tipos e papéis de permissão foram conferidos na documentação oficial do Drive. [S06]

Não mover os originais para TRATADAS como efeito do sucesso de mensagem. Uma operação de arquivamento separada precisa de recibo de backup, confirmação de integridade e atualização da origem rastreável. Não limpar BAIXA ou históricos automaticamente durante essa migração.

Retenção

Separar prazo de acesso da experiência, retenção de derivados, versões antigas e retenção de originais contratados. O texto legado menciona 90 dias; isso não foi adotado automaticamente como política nova. Definir `expiresAt` e política administrativa antes da publicação. Remoção do catálogo deve impedir reaparecimento por pastas antigas e considerar revisões acessíveis anteriormente.

18. Plano de implementação por entregas

Entrega	Trabalho concreto	Critério de conclusão
A. Identidade e intake	Parser do ID completo, origem raiz, bloqueio vazio, snapshot e resolução CRM com adapter fake.	5000 e IDs maiores preservados; BAIXA excluída; conflito/ambiguidade bloqueados.
B. Worker de mídia	CLI JSON versionada, recipeKey, namespaces, validação do snapshot e outputs.	30 fotos/90 variantes; retry sem recompressão; mudança de receita gera versão nova.
C. Ledger e lock	UUIDs, unicidade, leases/fencing, states transacionais e receipts.	Dois workers não ativam o mesmo build; resultado atrasado não vence o dono atual.
D. Backend de sessão	Persistência, autorização, token/cookie, API de catálogo e mídia.	Token inválido não mostra fixtures; cliente A não acessa foto B.
E. Galeria e jogos	Hidratação real, capa, revisão fixada, seleção e catálogo habilitado.	Galeria e jogos mostram as mesmas fotos do pedido correto em mobile.
F. Publicação e entrega	Upload privado com recibo, ativação atômica, outbox e CRM.	Lote parcial invisível; envio só após revisão ativa e destinatário confirmado.
G. Operação da temporada	Painel, alertas, recuperação, retenção e migração do legado.	Operador consegue explicar o estado de cada pedido e retomar com segurança.

A integração externa só deve ser validada em ambiente controlado após os contratos locais. Não adaptar silenciosamente os flows de produção em uso. O piloto inicial deve usar um pedido reconciliado e uma sessão aprovada, com rollback para a revisão anterior.

Testes necessários para implementação

1. Parsing de ID completo, zeros, ausência/ambiguidade e pasta com apóstrofo/acentos.
2. Raiz versus BAIXA, extensão mista, arquivo falso, link para fora da origem e raiz vazia.
3. Origem alterada com mesmo tamanho/mtime; corrupção de snapshot; foto acima dos limites.
4. Inclusão/substituição/remoção de foto sem renumerar outros IDs; duas versões da mesma captura.
5. Mesma receita reutiliza cache; receita diferente não retorna imagem antiga.
6. Falha de uma variante bloqueia coleção; manifestos incompletos não são ativos.
7. Interrupção entre escrita e promoção; dois workers; lease expirado e commit tardio.
8. Token/cookie inválido, expirado ou revogado; travessia de path; variante não permitida; acesso entre clientes.
9. Catálogo sem fixtures em produção; reentrada de jogo, mudança de foto hero e teardown.
10. Upload incompleto, checksum divergente, troca de revisão e rollback.
11. Timeout do envio, reconciliação e prevenção de reenvio cego.
12. Download/backup separados e nenhuma publicação do original por endpoint de jogos.

Esses testes são critérios do plano futuro. Não foram apresentados como testes já executados nesta auditoria.

19. O que foi validado nesta auditoria

Verificação executada	Resultado
Inventário dos arquivos principais	30 entradas JPEG na raiz; BAIXA fora da origem.
Hash dos originais antes/depois	Todos inalterados.
Geração com o worker existente	30 prontas; zero falhas; 90 variantes.
Decode dos derivados	90 arquivos decodificados com sucesso.
Orientação, proporção e metadados	Proporção dentro da tolerância; sem EXIF/XMP; sRGB.
Reexecução idempotente	30 prontas; zero alterações/regenerações nos outputs; 1,72 s.
Comparação de codecs	12 outputs de três amostras; bytes, tempo e PSNR registrados.
Galeria offline	30 cards, abertura, próxima foto, Escape e ausência de overflow/erro nos viewports 390, 768 e 1280 px.
Galeria React de referência	Build aprovado e smoke de UI em três viewports, com APIs/mídias simuladas e nenhum contato com o backend PHP real.
Sessões/CRM/produção	Apenas análise estática e desenho; não houve operação externa.

A galeria `GALERIA_LOCAL_VALIDACAO_5000.html` é um artefato offline de bancada com as fotos reais e derivados embutidos, sem acesso ao CRM, autenticação, envio ou hospedagem. Não é o backend de produção. Ela permite avaliar concretamente a coleção e o comportamento da galeria em tela pequena.

Pacote de evidências

Arquivo	Conteúdo
source-inventory.json	Foto, SHA-256 original, dimensões, bytes e flags de metadados; inventário BAIXA separado.
baseline-benchmark.json	90 saídas, hashes, dimensões, totals e tempo fresco.
baseline-replay-verification.json	Replay, decode e imutabilidade dos outputs.
compression-comparison.json	As 12 medições e método de PSNR.
study-summary.json	Resumos, catálogo dos jogos e projeções de capacidade.
gallery-verification.json	Verificação do protótipo em três larguras.
legacy-gallery-verification.json	Verificação da interface React do terceiro repositório com dados locais e APIs simuladas.
gallery-build.log	Build do clone festive-gallery-show; não equivale a typecheck/testes do novo port.
evidence/source-receipts.json	Snapshots de código, hashes, versões Git e estado de trabalho.
private-benchmark-storage/	Cache de originais e derivados da bancada; exclusivamente privado.
compression-samples/	Outputs de comparação local; não destinados à publicação.

A bancada e as medições podem ser reproduzidas pelos scripts preservados nesta pasta. Rodar a bancada novamente não significa publicar para o cliente. Código de produção, secrets, flows, filas e originais do cliente permaneceram fora das alterações desta auditoria.

20. Mapa de evidências locais

Os caminhos abaixo são relativos às raízes indicadas. Os números de linha correspondem aos arquivos inspecionados. Se o código mudar depois desta captura, consultar o snapshot preservado e seu hash.

EvydFlow - <repo-evydfLOW>

Código	Arquivo e linha	Evidência
E01	evydfLOW/tasks/fs.py:9, :34	Scan da raiz, prefixo e truncamento do pedido.
E02	tests/unit/test_fs.py:39	Teste que exige a regra legada dos quatro dígitos.
E03	evydfLOW/tasks/orders.py:20; evydfLOW/providers/evydencia.py:17	Detail por UUID; filtro numérico real ainda não comprovado.
E04	evydfLOW/tasks/images.py:56, :106, :131; evydfLOW/settings.py:54	BAIXA/QC, defaults e tolerância a falhas.
E05	evydfLOW/tasks/ftp.py:14, :68; flows/natal_default.yaml:35, :80	FTP e publicação direta; endereços legados.
E06	evydfLOW/tasks/drive.py:88; evydfLOW/drive/sharing.py:74	Cópia dos originais e permissão pública de edição.
E07	evydfLOW/flow/executor.py:135, :150; evydfLOW/flow/state.py:31, :49	Fingerprint e persistência/retomada.
E08	evydfLOW/queue/worker.py:67, :94; evydfLOW/tasks/whatsapp.py:57; evydfLOW/flow/executor.py:174	Execução, ACK/NACK e efeitos sujeitos a retry.
E13	evydfLOW/soclick/manifest.py:143, :154, :268; evydfLOW/soclick/storage.py:995, :1090, :1173, :1464; evydfLOW/soclick/worker.py:160	Manifesto, claims, reconciliação, gates e limites de reutilização.

Jogos - <repo-gameplay>

Código	Arquivo e linha	Evidência
E09	packages/platform/src/contracts/index.ts:30, :43; apps/play/src/phaser/createGame.ts:16	Contratos de Photo/Session e loading dos módulos.
E10	tools/media-pipeline/src/index.ts:68, :93, :144, :184, :226, :249	Namespace, limites, batch, manifesto, snapshot e derivados.
E11	apps/play/src/app/AppRouter.tsx:86; packages/platform/src/session/InMemorySessionRepository.ts:3; apps/catalog-server/src/CatalogServer.ts:37	Fixture fallback e lacunas da sessão/mídia de produção.
E12	tools/media-pipeline/src/prepareLocal.ts:8, :54, :66; tools/media-pipeline/src/inspect.ts	Ferramentas de teste local, IDs e forma de inspeção.
E14	apps/catalog-server/src/CatalogServer.ts:71	X-Accel aplicado a preview, dependente do proxy.
E15	packages/games/*/src/definition.ts	Mínimos, recomendadas e seleção do catálogo.
E16	packages/games/rena-das-lembrancas/src/domain/PhotoSelection.ts:3; packages/games/rena-das-lembrancas/src/runtime/phaser/createRenaDasLembrancasGame.ts:156, :726	Até oito fotos únicas e seleção de variantes de Rudolph.
E17	docs/media/NATAL_2024_CORPUS_BASELINE.md	Baseline histórico de cabeçalhos de 10.244 fotos; não reexecutado nesta bancada.
E18	apps/play/src/components/SessionPhotoAlbum.tsx:17; apps/play/src/components/PhotoPrint.tsx:11	Álbum compacto, picker e variantes compartilhadas.
E19	apps/play/src/styles.css:103; apps/play/src/shell.css:685	Fit genérico cover e específico contain no picker.
E20	apps/play/src/app/AppNavigation.ts:39; apps/catalog-server/src/CatalogServer.ts:97	Parsers que ainda precisam reconhecer a rota /fotos.
E21	packages/games/guirlanda-das-lembrancas/src/runtime/phaser/createGuirlandaDasLembrancasGame.ts:147; packages/games/expresso-das-fotos/src/domain/ExpressPhotoLoadPlan.ts:25	Variantes por papel e oportunidade de memória.

Galeria - hypeneural/festive-gallery-show, **commit** 0f903fba

Código	Arquivo e linha	Evidência
G01	src/pages/Index.tsx:14; src/App.tsx:20	Entrada por query id e roteamento separado.
G02	src/components/PhotoGallery.tsx:8, :45, :59, :168, :202, :260, :334	Contrato src único, lotes, API, observer, grid e lightbox.
G03	package.json; package-lock e manifests instalados	React 18 no app antigo; libs declarando peer React 19; build observado.
G04	src/components/GalleryHeader.tsx:10	Compartilhamento da URL atual; adaptar ao link canônico da sessão.

21. Referências técnicas primárias

As fontes externas servem para conferir comportamentos e princípios específicos; os achados do caso e as recomendações de integração foram inferidos do código e da bancada local.

- **S01 - Sharp, output e metadados:** comportamento default de sRGB e remoção de metadados; controles explícitos de ICC/EXIF/XMP. <https://sharp.pixelplumbing.com/api-output/>
- **S02 - Sharp, resize:** diferenças entre inside, contain, cover e withoutEnlargement. <https://sharp.pixelplumbing.com/api-resize/>
- **S03 - SQLite, WAL:** leitura/escrita concorrentes, um escritor por vez e restrição de WAL em filesystem de rede. <https://sqlite.org/wal.html>
- **S04 - OWASP, autorização:** negar por default e verificar permissões em cada requisição, incluindo recursos estáticos. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- **S05 - Caddy, reverse proxy:** interceptação de respostas e necessidade de configuração para servir resposta/arquivo alternativo. https://caddyserver.com/docs/caddyfile/directives/reverse_proxy
- **S06 - Google Drive, permissions:** tipos, papéis e efeito de allowFileDiscovery. <https://developers.google.com/workspace/drive/api/reference/rest/v3/permissions>

Nenhuma dessas referências comprova a configuração do servidor, o cadastro real do pedido 5000 ou permissões remotas efetivamente concedidas pelo estúdio.

22. Recomendação final para o pedido 5000

Usar as 30 fotos principais como seleção inicial; confirmar com o fotógrafo as duas versões da captura com duas versões e escolher capa/ordem. Preservar 5000 integralmente, resolver o UUID do pedido real e criar uma sessão fotográfica estável. Gerar/reutilizar thumb/card/game com a receita WebP 82 versionada. Manter o mesmo catálogo de fotografias para galeria e jogos, com seleção por jogo e foco na foto escolhida pela família.

Antes da entrega, implementar e validar hidratação real de sessão, autorização de mídia, transferência privada, revisão atômica, ledger e outbox. Enviar um link seguro da sessão somente depois de ativar a coleção validada e confirmar o destinatário do CRM. Originais, BAIXA legada e backup permanecem em seus ramos privados, sem movimentação implícita causada pela nova experiência.

O processamento de mídia foi comprovado. O que falta é integrar identidade, publicação e entrega de forma transacional e autorizada.

Anexo A. Inventário individual mantido privado

O inventário completo e os hashes das fotografias permanecem em armazenamento privado. A edição pública contém apenas contagens, dimensões agregadas e resultados técnicos em `evidence/benchmark-summary.json`. Nenhum original foi removido, renomeado ou sobrescrito.

Anexo B. Receita e fronteiras da bancada

```
{
  "referenceDate": "2026-10-05",
  "scope": "root files only, no BAIXA",
  "variants": {
    "thumb": {
      "maxLongEdge": 480,
      "format": "webp",
      "quality": 82
    },
    "card": {
      "maxLongEdge": 800,
      "format": "webp",
      "quality": 82
    },
    "game": {
      "maxLongEdge": 1600,
      "format": "webp",
      "quality": 82
    }
  },
  "resize": {
    "fit": "inside",
    "withoutEnlargement": true
  },
  "metadata": {
    "autoOrient": true,
    "outputColourSpace": "srgb",
    "exif": false,
    "xmp": false
  },
  "inputLimits": {
    "maxBytes": 33554432,
    "maxPixels": 40000000,
    "maxChannels": 5
  },
  "worker": {
    "batchConcurrency": 2,
    "sharpConcurrency": 1,
    "sharp": "0.35.3",
    "libvips": "8.18.3"
  },
  "publication": "not performed",
  "crmLookup": "not performed",
  "notifications": "not performed"
}
```

Plano de implementação - experiência fotográfica unificada

Edição pública anonimizada - 05/10/2026. O pedido 5000 e os nomes foto-NN.jpg são exemplos fictícios; não identificam o cliente da bancada. As medições agregadas permanecem reais. Fotografias, nomes, telefones, caminhos locais e hashes individuais de mídia foram omitidos. Artefatos privados mencionados na auditoria não acompanham esta edição; consultar evidence/benchmark-summary.json e evidence/source-index.json para os recibos públicos. A documentação registra fatos e propostas, não uma integração já implantada.

EvydFlow + gameplay + galeria mobile | 05/10/2026 | Versão 1.0

Este plano complementa a auditoria forense. É um roteiro de desenvolvimento e validação; os arquivos novos, rotas e interfaces abaixo são propostas, não funcionalidades já implementadas. O experimento com as fotos do pedido 5000 foi local e não publicou uma sessão de cliente.

P01. Resultado de produto e decisões de arquitetura

A família recebe um link seguro. Esse link abre um Hub compacto com a foto escolhida e os jogos; a galeria completa é acessível no mesmo aplicativo. As fotografias do pedido são processadas uma vez e reutilizadas nas três experiências:

Uma sessão fotográfica persistente	
/s/<token>	Hub e foto escolhida
/s/<token>/fotos	Galeria mobile completa
/s/<token>/game/<game-id>	Jogo com as fotos da mesma sessão

Decisões que orientam todo o trabalho

ADR	Decisão inicial	Justificativa e limite
ADR-01	Galeria dentro de apps/play, na gameplay.	Aproveita Photo, Session, PhotoPrint, tema e seleção existentes; uma API e um catálogo.
ADR-02	EvydFlow continua em Python/YAML.	Reaproveita operação do estúdio e providers; não reescrever o orquestrador.
ADR-03	Sharp gera os derivados no computador do estúdio no MVP.	Bancada comprovou 314,66 MB para 6,62 MB; transferir só derivados reduz tráfego.
ADR-04	API HTTPS de aplicação para ingestão e recibos.	Sessão/revisão/verificação não cabem no contrato de FTP legado.
ADR-05	Três variantes proporcionais, WebP 82.	480/800/1600 já medidas; não criar outra resolução sem necessidade de zoom comprovada.
ADR-06	Pedido completo + UUID CRM + UUID da sessão são identidades distintas.	Evita truncamento/colisão e não expõe o pedido como credencial pública.
ADR-07	Fonte da raiz, nunca BAIXA/QC para o novo pipeline.	Preserva detalhe e elimina recompressão/bordas do legado.
ADR-08	Revisões imutáveis e ativação transacional.	Galeria e jogos nunca veem uma coleção recebida pela metade.
ADR-09	Outbox de evento no backend; entrega operacional no EvydFlow.	EvydFlow já administra canais de comunicação; ativação gera evento durável, não mensagem imediata.
ADR-10	Port seletivo da galeria React antiga.	Aproveitar masonry/lightbox/zoom; adaptar mídia, autorização, tema e estado ao novo app.

Se for necessário gerar fotos na VPS posteriormente, criar modo separado source-ingest com quotas/receitas/recibos próprios. Ele não deve competir com prepared-derivatives pela mesma revisão. O worker é o mesmo pacote versionado, não uma segunda implementação de resize.

P02. Fronteiras dos três repositórios

Repositório	Responsabilidade a manter	Mudanças planejadas
hypeneural/gameplay	Contratos de Photo/Session, UI mobile, runtimes, pipeline Sharp e catálogo.	API de sessão/ingestão/autorização; galeria; revisão; melhorias de consumo de variantes.
EvydFlow em <repo-evydf>	Intake, pedido CRM, fila, retomada, providers e entrega do estúdio.	Identity/ledger de experiência, adapter de mídia, IGameplayProvider, novo flow e outbox de entrega.
hypeneural/festive-gallery-show	Referência visual e funcional da galeria React.	Extraí/adaptar componentes e padrões; manter legado durante migração. Não criar outra autoridade de mídia.

Cada novo arquivo listado neste plano é um local sugerido. Ajustar nomes à estrutura definitiva sem misturar código de banco/autorização com pacotes de frontend. Não copiar os manifests/lockfiles e toda a árvore shadcn/Tailwind da galeria para o monorepo.

Ordem das dependências

```
M0 contratos e identidade
-> M1 worker de mídia versionado
-> M2 persistência e revisão do backend
-> M3 ingestão/verificação/ativação HTTPS
-> M4 acesso público autorizado e hidratação real
-> M5 galeria na gameplay
-> M6 provider e flow do EvidFlow
-> M7 evento/outbox e entrega
-> M8 otimização de jogos e QA
-> M9 piloto e migração
```

Fixtures e testes de contrato podem ser desenvolvidos em paralelo com o backend. Publicação real depende das garantias de revisão/autorização; não começar por alterar o flow que já envia mensagens.

P03. M0 - identidade e contrato comum

Objetivo: um número completo de pedido aponta para uma sessão estável, sem conflito com os identificadores de mídia/acesso.

Trabalho no EvidFlow:

- Criar parser de intake do sufixo - <dígitos> que retorne `crm_order_number` completo como string.
- Criar `orders.validate_photo_session` ou equivalente: conferir pedido por UUID, tenant, número completo, titular e destinatário.
- Aproveitar `--order-uuid`, já aceito pelo `natal_uploader.py`, como entrada do MVP. O valor hoje apenas entra em `params`; o novo flow precisa realmente usá-lo e validá-lo.
- Encaminhar `OrderUuid/ExtraParams` no runner PowerShell e nos atalhos por array de argumentos, sem concatenação de shell.
- Automatizar número para UUID somente depois de confirmar o filtro real de `/orders/search`. O fake atual ignora o filtro e não prova essa busca.
- Persistir `photo_session_id` UUID e `session_key` de negócio. Uma nova execução do mesmo intake reutiliza a sessão.
- Não trocar o significado do extrator legado para todos os flows sem migração dos diretórios/links antigos; o novo fluxo não usa a regra dos quatro dígitos.

Trabalho compartilhado de contrato:

- Definir um schema versionado de ingestão e recibos, com fixtures para TypeScript/Zod e Python/Pydantic.
- Definir `PublicPhotoSessionDto` separado do manifesto privado de origem.
- Acrescentar revisão, capa e jogos habilitados à projeção pública.
- Manter `Photo.variants` como URLs para compatibilidade dos runtimes; adicionar metadados de variante com largura/altura/bytes quando necessários à galeria.
- Padronizar códigos de erro e estados, sem retornar paths, tokens ou dados completos do CRM nas mensagens de UI.

Casos de aceite

5000, 105000 e 1004821 preservados; pasta inválida bloqueada; UUID explícito incompatível bloqueado; pedido ambíguo bloqueado; telefone da pasta diferente do destinatário CRM abre conferência; dois intakes do mesmo pedido não sobrescrevem sessões diferentes silenciosamente.

P04. M1 - worker Node/Sharp e manifesto de conteúdo

Responsável: gameplay, em tools/media-pipeline, com adapter operacional no EvidFlow.

Arquivos e capacidades planejados

Local sugerido	Capacidade
tools/media-pipeline/src/contrasts.ts	Job JSON versionado, limites, seleção e recibo.
src/recipes.ts	Receitas explícitas e recipeKey canônica.
src/intake.ts	Snapshot congelado, SHA-256 e allowlist.
src/batchCli.ts	CLI de coleção com stdout JSON e progresso separado.
src/index.ts	Reaproveitar processamento existente, incluindo receita no namespace.
tests/	Conteúdo alterado, receita alterada, crash, falha parcial e replay.
Release do worker	Executável/runtime fixos e distribuição reproduzível para o estúdio.

Selecionar só arquivos diretos na raiz. Normalizar extensão e validar conteúdo; não seguir symlink/junction que leve para fora da origem aprovada. Verificar estabilidade da exportação e impedir zero fotos. Não assumir que todas as sessões têm 30 ou 32 fotos.

Copiar para snapshot temporário; recalcular hash do snapshot e comparar com o original congelado; promover o snapshot só após conferência. O SHA-256 do pipeline é dos bytes do arquivo, não um hash de reconhecimento/pixels. Trocar metadados pode mudar o hash mesmo quando a aparência é semelhante.

IDs das fotos são persistentes, não uma numeração reconstruída da ordenação atual. Incluir uma foto antes da primeira não deve mudar o ID das outras. Hash igual pode permitir reaproveitar bytes dentro do escopo autorizado, sem fundir duas escolhas de curadoria ou misturar clientes.

Receita inicial

```
source snapshot verificado
-> orientação EXIF normalizada
-> sRGB, sem EXIF/XMP de origem
-> inside, sem ampliar
-> thumb: lado maior 480, WebP 82
-> card: lado maior 800, WebP 82
-> game: lado maior 1600, WebP 82
```

Propor namespace derived/<session>/<photo>/<sourceHash>/<recipeKey>/<variant>. O hash de receita inclui parâmetros, versão do worker/encoders e política de metadados. O cache atual usa conteúdo sem receita; essa lacuna deve ser corrigida antes de oferecer reproprocessamento com nova qualidade.

A escrita do conjunto ocorre em staging no volume de destino. O manifesto é uma revisão completa, com seleção explícita; não fazer merge que conserve fotos removidas por engano. Leases/fencing ou escritor único protegido impedem dois processos de promover o mesmo namespace simultaneamente.

Aceite e prova mínima

Usar o pedido 5000 como fixture privada: 30 fontes, 90 derivados, proporção preservada, nenhum EXIF/XMP, originais intactos. Replay da mesma receita não altera outputs; receita diferente cria namespace novo. Falha em qualquer variante bloqueia a coleção por default. A bancada existente mediu geração em 21,75 s e replay em 1,72 s; essas medições são referência, não limite universal de CI.

P05. M2 - persistência, revisões e autoridade do backend

Responsável: gameplay. Proposta inicial: evoluir apps/catalog-server com módulos próprios de sessão/ingestão/mídia. O nome atual não deve obrigar a criar um segundo serviço de autenticação.

Locais sugeridos:

```
apps/catalog-server/src/
  experience/contracts.ts
  experience/SessionService.ts
  experience/SessionRepositorySqlite.ts
  experience/RevisionRepository.ts
  access/AccessGrantService.ts
  access/ServiceAuthentication.ts
  media/AuthorizedMediaDelivery.ts
  ingest/PreparedMediaRoutes.ts
  ingest/RevisionActivation.ts
  events/DomainOutboxRepository.ts
```

Os nomes são planejamento. Banco, credenciais e filesystem permanecem no backend; packages/platform recebe apenas contratos públicos necessários aos consumidores, sem drivers SQL, secrets ou dependência de Node no navegador.

Modelo de dados mínimo proposto

Entidade	Dados e garantias essenciais
photo_sessions	UUID, tenant, UUID CRM, número completo, sessionKey, estado, activeRevision, validade.
revisions	UUID, sessão, fingerprint, receita, estado, contagem e recibo; unicidade por sessão/fingerprint.
revision_photos	Ordem, capa, photold estável e metadados normalizados aprovados.
media_files	Sessão/revisão/foto/variante, namespace privado, dimensões, MIME, bytes e hash.
access_grants	Hash do token, vínculo com sessão, expiração, revogação e escopo.
browser_access_sessions	Cookie opaco e concessões autorizadas; não usar o número do pedido como sessão de navegador.
domain_outbox_events	Evento durável PHOTO_SESSION_ACTIVE criado com a ativação da revisão.
audit_events	Ação, identidade operacional opaca, versão e resultado; sem token/telefone/path nos logs gerais.

SQLite/WAL em disco local do host é uma opção inicial. Decidir o driver compatível com o runtime Node e testar contenção/backup. Não compartilhar o arquivo de banco entre Windows/VPS via Drive, SMB ou sync de pastas. Um contrato HTTPS sincroniza recibos e eventos; não duplica a autoridade de sessão.

Transação de ativação

Exigir revisão VALIDATED, conjunto exato de variantes recebidas, recibo de hashes, lease/fencing válido e expectedActiveRevisionId. Atualizar o ponteiro ativo e inserir o evento de domínio na mesma transação. Um cliente/worker atrasado recebe conflito, sem sobrescrever a revisão mais recente.

Access token é aleatório independente do pedido. Armazenar hash para verificação; caso a entrega precise reenviar a mesma URL, proteger o payload de entrega com armazenamento cifrado/keystore do serviço. Hash irreversível,

sozinho, não permite reconstruir o token original. Nunca guardar a URL com token em logs ou manifesto público permanente.

P06. M3 - contrato HTTPS prepared-derivatives

Responsável: backend gameplay e provider EvydFlow. As rotas abaixo são propostas, com prefixo/versionamento ajustável na implementação.

Operação proposta	Entrada / resultado esperado
POST /internal/v1/photo-sessions	Referência CRM/sessão e chave idempotente; retorna sessão existente ou nova.
POST /internal/v1/photo-sessions/:id/revisions	Manifesto congelado, recipeKey, seleção e descrição dos 90 arquivos.
PUT /internal/v1/revisions/:id/media/:photoId/:variant	Stream, tamanho e hash; aceita somente variante/receita permitida.
POST /internal/v1/revisions/:id/verify	Confere inventário remoto exato e produz recibo.
POST /internal/v1/revisions/:id/activate	Condição sobre revisão ativa anterior + chave idempotente.
GET /internal/v1/photo-sessions/:id/status	Estado da revisão/fingerprint, contagens e causa de bloqueio.
GET /internal/v1/events e ACK	Consumo durável de eventos pelo EvydFlow, com cursor/ID único.

Recibo proposto para a coleção 5000

```
{
  "schemaVersion": 1,
  "sessionId": "uuid-da-sessao",
  "revisionId": "uuid-da-revisao",
  "status": "VALIDATED",
  "manifestFingerprint": "sha256-do-manifesto",
  "recipeKey": "sha256-da-receita",
  "photos": { "expected": 30, "ready": 30, "failed": 0 },
  "files": { "expected": 90, "verified": 90, "failed": 0 },
  "derivedBytes": 6616226,
  "hashesVerified": true
}
```

VALIDATED não é ACTIVE; ACTIVE não é DELIVERED. O operador e o provider precisam distinguir os três estados.

Autenticação de serviço, HTTPS, escopo por tenant e limites de corpo/arquivos são obrigatórios. O backend não recebe paths Windows para abri-los, não busca URLs arbitrárias do produtor e não serve originais. Ele valida os bytes derivados recebidos; o hash original é proveniência do produtor, porque o original não foi transferido no MVP.

O prefixo /internal não é um controle de acesso. Proteger o ingresso e a credencial de serviço de fato; se houver rede privada/VPN ou allowlist, configurar e testar também essa camada. Não expor a API genérica /jobs do EvydFlow como substituto da ingestão, nem permitir que um cliente remoto escolha flow YAML ou pasta do Windows.

Por replay, mesma chave e mesmo payload retornam o recibo anterior; mesma chave com payload diferente retorna conflito. Upload de variante já verificada com hash diferente é conflito, não sobrescrita. Só contar foto READY

quando todas as suas variantes existirem e passarem validação. Validar arquivo WebP, limites, dimensões, proporção, metadados e hash, mesmo quando a origem é um serviço confiável.

Arquivos pequenos podem ser transferidos individualmente com conexão reutilizada e concorrência limitada. ZIP de ingestão não é necessário no MVP; se adicionado depois, precisa de validação de conteúdo, limites e proteção contra path traversal/expansão excessiva.

Critério de saída

Um teste com fake e um teste de integração local devem provar lote incompleto invisível, checksum divergente bloqueado, replay sem duplicação, timeout de transferência retomável e ativação condicionada à revisão anterior. Nenhum callback de WhatsApp é disparado pela rota de upload.

P07. M4 - acesso da família e sessão real no React

Responsável: gameplay.

Criar `HttpSessionRepository` e um `SessionExperienceProvider` acima de Hub, galeria e jogos. A borda resolve o token, verifica acesso e carrega a projeção autorizada. Em produção, token inválido/expirado/revogado deve mostrar uma tela apropriada, nunca fixtures.

Locais sugeridos:

```
apps/play/src/app/SessionExperienceProvider.tsx
apps/play/src/app/HttpSessionRepository.ts
apps/play/src/app/SessionResponseSchema.ts
apps/play/src/app/AppNavigation.ts
apps/play/src/app/AppRouter.tsx
apps/catalog-server/src/access/PublicSessionRoutes.ts
apps/catalog-server/src/media/AuthorizedMediaRoutes.ts
```

API pública e escopo entre abas

Uma troca de token por acesso retorna `sessionId` autorizado. Preferir a consulta explicitamente escopada, como `GET /api/v1/photo-sessions/:sessionId`, em vez de depender de um "current" global ambíguo. Toda resposta é conferida contra o `sessionId/revisionId` esperado pelo provider.

Se o browser puder abrir duas sessões em abas distintas, o cookie/registro de navegador precisa representar concessões autorizadas e cada request deve indicar/validar o objeto solicitado. Uma nova aba não pode fazer a aba anterior mostrar a sessão errada. A política de uma única sessão por navegador também é possível, desde que resulte em bloqueio claro, não troca silenciosa de fotos.

`Photo.variants` recebe URLs relativas autorizadas. O DTO deve informar dados aprovados, não fields integrais do CRM. Para compatibilidade temporária com `Session.publicToken`, a borda pode associar o token de entrada já presente na rota após resolver a sessão; não usar 5000 no lugar dele nem incluir tokens em telemetria.

Estados e navegação

Loading, ready, unavailable, expired, revoked, empty e retryable-error. `AbortController` e identificador de requisição descartam respostas tardias de outro token. Ao trocar de cliente, limpar seleção, lightbox e recursos do cliente anterior. Dentro da mesma sessão, preservar foto escolhida e scroll ao alternar Hub/galeria/jogo.

Uma rodada fixa a revisão usada no início. Atualizações do catálogo não remapeiam pares/cartas enquanto um jogo está em andamento. O mecanismo existente de saída do Phaser continua responsável por desmontar a instância antes de mudar de tela; não remountar o canvas por uma atualização de mute, seleção de UI ou request tardio.

Media gateway

Autorizar cookie/concessão, sessão, photold, revisão e variante em cada request. Servir por streaming do backend no MVP ou por proxy interno configurado e testado. X-Accel-Redirect depende do Nginx ou de interceptação explícita no proxy; não é uma autorização automática do Caddy.

Recusar path traversal, MIME/extensão falsa, acesso entre clientes e variantes fora do contrato. Proteger originais fora da webroot. HTML, manifestos e fotos privadas não entram em cache compartilhado ou service worker indiscriminado. Compartilhamento social usa arte genérica por default; crawler não deve consumir o token de acesso de forma definitiva.

P08. M5 - galeria mobile dentro da gameplay

Responsável: apps/play. Referências principais: festive-gallery-show/src/components/PhotoGallery.tsx, GalleryHeader.tsx, SessionPhotoAlbum.tsx e PhotoPrint.tsx da gameplay.

Arquivos/componentes planejados

Local sugerido	Responsabilidade
screens/SessionGallery.tsx	Coleção completa, grid, paginação de UI e estado de retorno.
components/SessionLightbox.tsx	Foto completa, gestos, navegação, zoom e foco.
components/GalleryPhotoTile.tsx	Espaço proporcional reservado e variante do grid.
components/PhotoPrint.tsx	Acrescentar fit explícito e metadados responsivos, mantendo compatibilidade.
app/AppNavigation.ts	GalleryRoute e URL /s/:token/fotos.
app/AppRouter.tsx	Rota ligada ao provider comum, sem outra origem de fotos.
gallery.css ou módulos equivalentes	Estilo com escopo e tokens do shell, sem override global de todos os jogos.
CatalogServer/parser de rotas	HTML/social/reload para /fotos, além de Hub e jogos.

O texto colado recomenda reaproveitar os conceitos da galeria antiga. O port adaptará esses componentes/padrões ao contrato comum da sessão. As libs instaladas declaram React 19, mas devem passar typecheck, arquitetura, bundle e testes no ambiente React 19/TypeScript do monorepo. Importar lightbox/zoom sob demanda e não carregar Phaser na rota de fotos.

Contrato visual

Foto hero e lightbox: contain, sem deformação. Grid proporcional pode usar thumb ou card conforme largura e densidade. Miniatura cover só quando houver enquadramento confiável; um único focalPoint não comprova proteção de todas as pessoas. Se a caixa segura não couber no crop calculado, usar contain.

Usar classes/atributo de fit explícito, por exemplo data-photo-fit="contain"; eliminar dependência acidental de precedência entre .photo-card img genérica e regras locais. Ponto focal e faceSafeZone se referem à imagem orientada, não à câmera antes da rotação EXIF. Não executar reconhecimento biométrico para escolher a capa.

Detalhe importante de srcset: 480/800/1600 são limites do lado maior. Em foto vertical, a largura real pode ser 343/571/1143. Não escrever 480w/800w/1600w simplesmente usando o nome da variante. Os descritores devem usar as dimensões reais do derivado; sizes deve representar o espaço CSS do grid. Esse dado precisa constar no manifesto de variantes.

Carregamento progressivo

Começar com lote pequeno (referência antiga: 12 mobile/18 desktop) e incremento limitado, com IntersectionObserver e botão "Ver mais" como fallback acessível. Reservar aspecto antes do decode para não empurrar o sentinel e disparar todo o acervo. Definir e testar máximo de antecipação de fotos e preloads do lightbox.

Grid pequeno: thumb/card. Se uma fotografia ocupar grande parte da tela, a densidade e largura real podem justificar game também nessa visualização, carregada somente quando visível. Lightbox: game selecionada e poucos vizinhos. Não baixar game para todas as fotos só porque os slides foram criados. No teste de referência, a galeria tinha variáveis de lote, mas já havia renderizado 28/30 fotos no momento observado; o novo orçamento precisa ser demonstrado por requests/visibilidade e não só por código de slice.

Reaproveitamento entre galeria e jogos pode usar um resolver limitado de recursos em memória, sempre com chave de sessão/revisão/foto/variante. Se usar blobs/Object URLs, definir dono, limite e liberação ao sair/trocar de sessão. Isso é diferente de cache persistente de fotos privadas; não prometer reutilização de rede somente porque duas tags usam a mesma URL com no-store.

Interação

Anterior/próxima, swipe, Escape, foco restaurado, close visível e pinch/zoom compatível com navegação. Gestos devem distinguir arrasto de zoom de fechar/avançar; não fechar a foto involuntariamente ao tocar para interagir. Ações principais seguem os tamanhos de toque do projeto; controles do legado de 36 px não devem ser copiados automaticamente.

CTA "Brincar com esta foto" define selectedPhotoid no provider e retorna ao catálogo/entrada de jogo preservando essa foto. O Hub permanece compacto; não colocar toda a masonry acima dos jogos. Compartilhar/copiar usa URL canônica por token. Download do original é produto/ramo de entrega separado, não consequência de um botão de lightbox.

Aceite mobile

390/412/430/768 px, desktop e aparelhos reais; 1, 5, 30, 92, 120 e 172 fotos; orientação mista; rede lenta; imagem com erro; observer indisponível; reduced motion; retorno por back/reload; foco e gestos. Nenhum crop indevido de foto hero, scroll horizontal, mistura de clientes ou download de todos os originais. Os três viewports do protótipo desta auditoria não substituem essa matriz do port.

P09. M6 - provider e flow do EvydFlow

Responsável: EvydFlow, mantendo legado e perfis test/dry-run.

Arquivo/local sugerido	Alteração
evydfLOW/providers/interfaces.py	Protocolo IGameplayProvider e tipos de recibo.
providers/gameplay.py	Cliente HTTPS com autenticação, limites e erros tipados.
providers/fakes.py e factory.py	Fake completo e composição conforme perfil.
tasks/gameplay.py	Sessão, revisão, transfer/verify/activate e recuperação.
tasks/media_experience.py	Wrapper do worker de mídia, stdout JSON e cancelamento.
tasks/orders.py ou módulo dedicado	Validação de UUID/número completo/destinatário.
tasks/__init__.py	Registro explícito das novas tasks.
flows/photo_experience_publish.yaml	DAG independente de BAIXA/QC, Drive e mensagens imediatas.
settings / exemplo de config	API base, referência de credential, worker/release, roots e limites; secrets fora do Git.
runner/atalhos	Passar OrderUuid e flags do novo flow sem concatenar comandos.

Interface proposta

```
IGameplayProvider
  resolve_or_create_session(crmIdentity, sessionKey, idempotencyKey)
  begin_revision(sessionId, buildFingerprint, manifest)
  upload_derivative(revisionId, photoId, variant, stream, sha256, length)
  verify_revision(revisionId) -> receipt
  activate_revision(revisionId, expectedActiveRevision, idempotencyKey)
  get_revision_status(sessionId, buildFingerprint)
  read_domain_events(cursor)
  acknowledge_domain_event(eventId)
```

Esse protocolo não existe atualmente. Não encaixá-lo no IFtpProvider, que não representa sessão, conteúdo, revisão, autorização ou recibo.

Novo DAG

```
scan_root
-> validate_crm_identity
-> resolve_stable_photo_session
-> freeze_content_manifest
-> generate_sharp_derivatives
-> validate_complete_local_revision
-> begin_remote_revision
-> transfer_derivatives
-> verify_remote_receipt
-> activate_revision
-> reconcile_ready_event
```

order_uuid explícito pode destravar o MVP sem inventar filtro de pesquisa. A origem raiz permanece intacta. Não rodar simultaneamente o novo freeze e um flow legado que move essa mesma origem; freeze deve produzir snapshot completo antes de uma eventual operação de arquivamento.

Retries de transferência/verificação usam idempotência e são domain-managed, sem multiplicar retries do motor e provider indiscriminadamente. Falhas de schema, identidade, fonte alterada e checksum não são tratadas como falhas transitórias de rede. Timeout de operação externa vira resultado desconhecido até consulta de status/recibo.

Por desenvolvimento, cumprir o gate de verificação do EvidFlow com perfil de teste/dry-run, providers fake e sem WhatsApp/FTP/Drive reais. A auditoria atual não realizou a implementação nem executou esse gate para uma alteração de código inexistente.

P10. M7 - evento de ativação, inbox e entrega

O backend é autoridade de ACTIVE. EvidFlow é o executor dos canais já configurados do estúdio. A ponte tem dois registros com finalidades diferentes, sem dois emissores enviando a mesma mensagem:

```
Backend: ativar revisão + domain outbox, numa transação
-> EvidFlow lê evento PHOTO_SESSION_ACTIVE
-> persiste inbox por eventId + intenção de entrega local
-> ACK do evento somente depois da persistência
-> dispatcher usa o provider de comunicação configurado
-> registra providerMessageId / SENT / UNKNOWN_OUTCOME / FAILED
```

EventId único impede que repetir a leitura gere outra intenção. O payload mínimo leva IDs opacos, revisão e contagens; a URL de acesso/destinatário são consultados/protegidos conforme o contrato administrativo. Não colocar telefone/token no log geral.

Criar DeliveryIntent com chave por intenção de negócio, sessão, revisão, destinatário autorizado e versão de mensagem. Publicar uma revisão nova não deve obrigatoriamente reenviar comunicação: capa/ordem pode mudar sem nova intenção. Definir se o evento pede notificação ou apenas atualização operacional.

Se o PC fechar após o backend ativar, o evento permanece consumível. Se o provedor aceitar a mensagem e a rede cair, marcar UNKNOWN_OUTCOME e reconciliar antes de repetir. Não prometer exatamente uma vez sem suporte do provedor; o projeto deve demonstrar prevenção de duplicação controlável e tratamento da incerteza.

Registrar status da galeria, status da entrega e vínculo no CRM separadamente. Não mudar o status comercial do pedido só porque um upload concluiu. Escolher campos/endpoints reais do CRM após prova de contrato; nenhum campo de URL/status foi inventado como existente nesta auditoria.

P11. M8 - memória dos jogos e proteção da fotografia

Esta entrega é complementar à sessão/galeria; não bloqueia a construção do provider fake, mas precisa de aceite antes de habilitar jogos afetados para clientes.

- Guirlanda: preload da hero em game e demais em card; promoção sob demanda; liberar game anterior quando não tiver consumidor. Proteger callbacks após teardown/cancelamento.
- Usar como referência os planos de variante já existentes em Expresso, Rudolph e Mosaico.
- Preservar contain e proporção das fotografias, incluindo fotos horizontais dentro da tela vertical.
- Garantir que rodada fixa revisão e IDs; repetições em Memória/Rudolph são entidades de gameplay, sem recompressão ou duplicação no álbum da sessão.
- Animações/partículas devem respeitar zonas seguras e configurações de reduced motion/qualidade.
- Conferir o aviso de máscara WebGL da Lanterna registrado na auditoria anterior antes de afirmar recorte visual correto em toda a celebração. Essa mecânica não foi alterada neste trabalho.

Aceite: requests por variante observáveis, texturas limitadas, canvas destruído e retorno à galeria com foto escolhida preservada. A redução de 62,5% de área de pixels na Guirlanda é apenas estimativa para seis fotos equivalentes; medir memória/FPS em hardware antes de anunciar ganho real.

P12. Matriz de verificação integrada

Categoria	Casos que precisam passar	Evidência esperada
Identidade	IDs completos, UUID CRM explícito, conflito, tenant e destinatário.	Tests de parser/adapter e recibo de vínculo.
Origem	BAIXA ignorada, raízes permitidas, fake file, fonte mudando e zero fotos.	Manifesto e teste de segurança de intake.
Mídia	Receita nova, replay, foto substituída/removida, erro de variante.	Hashes/bytes/decode e versionamento.
Concorrência	Duplo job, lease vencido, crash, escritor atrasado.	Apenas uma ativação válida; fencing rejeita dono antigo.
API interna	Upload parcial, hash diferente, replay e timeout.	Recibo exato ou bloqueio tipado, sem side effect de entrega.
Acesso	Token/cookie inválido, expiração, revogação, cliente A/foto B.	Nenhuma leitura cruzada e nenhuma fixture em produção.
Roteamento	/s, /fotos, /game, reload, back, abas e troca de token.	Mesma sessão correta em todas as telas.
Galeria	Loading, erro, lightbox, zoom, swipe, observer indisponível e rede lenta.	Screenshots, requests e controle de antecipação.
Jogos	Mínimo de fotos, seleção, revisão fixa, áudio/pausa/teardown.	Testes existentes + jornadas da sessão real aprovada.
Entrega	Evento repetido, crash antes do ACK, timeout após envio.	Inbox/outbox sem reenvio cego e status reconciliado.
Privacidade	Originais/path/telefone/UUID CRM no DTO e cache compartilhado.	Projeção mínima e endpoint privado.
Migração	ID antigo, tokens novos, rollback e cliente já em rodada.	Nenhuma nova credencial revelada por ID enumerável.

No gameplay, usar typecheck, lint, unit, arquitetura, deadcode, format, repo map, assets, build e E2E pertinentes. No EvidFlow, usar seus gates em perfil de teste. Testes com provider fake não comprovam contrato/live behavior do provedor; a etapa externa controlada tem recibo próprio.

P13. M9 - piloto, migração e rollback

Primeiro corte vertical

1. Backend local com uma sessão de teste e três variantes por foto.
2. /s/:token e /fotos consomem essa sessão autorizada, sem fixture fallback.
3. Galeria escolhe foto; Puzzle entra com a mesma photold; saída preserva seleção.
4. Repetir com Memória e Rudolph para subconjunto/repetição/álbum.
5. Provider fake EvidFlow gera manifesto e simula upload/verificação/ativação.
6. Conectar a bancada real 5000 somente em storage/ambiente privado, sem envio ao cliente.

7. Resolver UUID CRM/destinatário reais por contrato validado antes do piloto comercial.

Os IDs posicionais photo-001 da bancada são de teste. Não usá-los como estratégia persistente de produção que renumera a coleção em cada scan.

Piloto comercial controlado

Sessão aprovada, autorização testada, backup validado, operador confirma pedido/destinatário e revisão ACTIVE. Primeiro envio usa outbox e recibo. Acompanhar abertura, erro de mídia e suporte sem registrar dados sensíveis. Habilitar apenas jogos que passaram QA no ambiente de destino.

Migração do legado

Manter festive-gallery-show, BAIXA/QC e flows antigos durante transição explícita. Novas sessões aprovadas passam pelo novo flow; legado não muda de comportamento sozinho. O fotógrafo não precisa exportar uma cópia por jogo.

Não fazer redirect público de um ID numérico antigo para um token secreto novo. Isso tornaria o token descobrível pela mesma enumeração que se deseja evitar. Reenviar novo link por canal/destinatário verificado ou exigir acesso autenticado antes de revelar a nova sessão. Uma landing informativa sem fotos pode explicar a migração.

Rollback

Backend troca activeRevision para a versão anterior por operação autenticada/auditada; não reescreve arquivos da revisão. App pode desabilitar GalleryRoute/jogos novos por feature flag sem apagar sessões/fotos. Job de entrega pode ser pausado independentemente da galeria. Revogação por privacidade é imediata conforme a política, sem reativar foto removida ao fazer rollback.

Não usar exclusão de data/state, limpeza indiscriminada de pasta ou force push como rollback operacional. Receipts/ledger e revisões permitem recuperação rastreável.

P14. Backlog executável, tamanho e dependências

Tamanhos são relativos, não promessa de dias: S = alteração localizada; M = módulo/fluxo com testes; L = contrato entre sistemas e validação integrada.

ID	Trabalho	Repositório	Depende	Tamanho / aceite
T01	ADRs, schemas e fixtures comuns	gameplay + EvydFlow	Nenhum	M; contrato versionado validado nos dois runtimes.
T02	Parser completo e vínculo CRM por UUID	EvydFlow	T01	M; zero truncamento/ambiguidade em novo flow.
T03	CLI Sharp, recipeKey e snapshot	gameplay	T01	L; replay/receita/crash/90 derivados corretos.
T04	Adapter Python do worker	EvydFlow	T03	M; processo sem shell e resultados validados.
T05	Banco/revisão/acess grants	gameplay	T01	L; constraints/leases/ativação transacional.

ID	Trabalho	Repositório	Depende	Tamanho / aceite
T06	Ingestão HTTPS preparada e recibos	gameplay	T03,T05	L; lote incompleto nunca ACTIVE.
T07	API de leitura/mídia autorizada	gameplay	T05	L; acesso cruzado negado e original inacessível.
T08	SessionExperienceProvider e rotas	gameplay	T07	M; token errado não mostra fixture ou outra sessão.
T09	Port galeria/light box/fit/variantes	gameplay	T08	L; mobile, foco, zoom e orçamento comprovados.
T10	IGameplayProvider real/fake	EvvdFlow	T01,T06	M; replay, conflito e recuperação.
T11	Flow independente + runner	EvvdFlow	T02,T04,T10	M; test/dry-run sem efeitos externos.
T12	Domain event, inbox e entrega	ambos	T05,T10,T11	L; crash/repetição/UNKNOWN_OUTCOME tratados.
T13	Otimização Guirlanda e QA de foto	gameplay	T08	M; consumo seletivo e teardown.
T14	QA integrado, piloto e migração	ambos + legado	T09,T12,T13	L; evidência de ponta a ponta e rollback.

Contrato CRM de número para UUID, detalhes de implantação do proxy e capacidade de hardware são pendências reais. Podem impedir automatização/produção, mas não impedem desenvolver o corte vertical com UUID explícito e providers fake. Não estimar calendário fechado sem resolver essas entradas e medir o time de implementação.

P15. Condição de conclusão e entregáveis

A implementação está concluída quando uma pasta selecionada da raiz, vinculada ao pedido completo correto, gera uma revisão verificada; o backend ativa essa revisão; um único link autorizado abre galeria e jogos com as mesmas fotos; e EvidFlow registra entrega com tratamento de retry/resultado desconhecido.

Entregáveis esperados: schemas; worker versionado; ledger/revisões; API interna e pública; galeria integrada; provider real/fake; novo YAML; outbox/inbox; roteiro de operação; evidências mobile; plano de migração e rollback. Código, docs e gates acompanham cada entrega.

O que já existe como evidência deste estudo

- Auditoria dos três repositórios em commits identificados.
- 30 fotos principais do pedido 5000 processadas, com 90 variantes e originais íntegros.
- Replay sem alterar derivados; comparação de compressão em três fotos.
- Galeria offline da coleção real e galeria React de referência verificadas localmente.
- Build da galeria antiga aprovado, sem contato com endpoints reais de cliente.

O que não foi implementado por este plano

SessionRepository de produção, GalleryRoute da gameplay, APIs novas, IGameplayProvider, novo flow, resolução CRM real, outbox e publicação comercial. O objetivo desta entrega é tornar a execução concreta e revisável, com bases técnicas e gates verificáveis, sem confundir uma bancada bem-sucedida com integração de produção concluída.